day / date

Algorithm: Effective Sequence Computational Steps. DP ✓ Greedy ✓
Works Every Input Instance!
Halts at Correct Output!

```
for(int i=0; i<n; i++)
    for(int j=0; j<n; j++)
        cout << "hello"; N
```

Inner loop

$N(3N+2) + N+(N+1)+1$

$T(N) = 3N^2 + 4N + 2$

$1+2+3 \dots n \Rightarrow \frac{n(n+1)}{2}$

```
for (int i=0; i<n; i++)
    for (int j=0; j<i; j++)
        cout << ...
```

$1 + N+1 + N + N \dots$ (3N+2)

$2\left(\frac{n(n+1)}{2}\right) + \frac{n(n+1)+n}{2}$

$j<i$

$\frac{1}{+1} + \frac{2}{+1} + \frac{3}{+1} + \dots + \frac{n}{+1}$

Since Condition was once more

Finally BingO!

day/date

Since 1 more Condition

$1 + (1 + \log N) + 1 + 2(1 + \log N)$

```
h=1  1
while (h<=n)
{
    h = h * 2;
    cout <<...
}
```

1 + 1 → $\log_2(2)$
1 + 2 $\log_2(4)$
1 + 3 $\log_2(8)$
1 + 4

```
for (j=1; j<=n; j++)
    for (k=n; k>=5; k--)
        for (i=1; i<=5; i++)
            --- cout
```

j=1	j=2	j=3	---	j=n
n	n-1	n-2		5
5n	5(n-1)	5(n-2)		5(1)

2	3	4	6	1

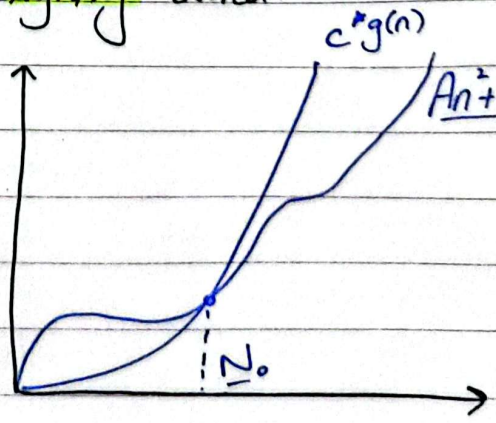
Best Case → O(1)
Avg Case → large values
Worst-Case → O(n)

Asymptotic Complexity:

Big Oh Notation. → Upper bound.
Omega Notation. → Lower bound.
Theta Notation. → Tightly bound.

g(N) => Higher Order Term

*After N_0, $Cg(N)$ always greater than $f(N)$ so Upper bound.



Finally BingO

day/date

Insertion Sort :- No tight Bound.

$[O(n^2)][\Omega(n)]$

→ g(n) should be of some degree for $\underline{O(n)}$ (theta)!

Binary Search:

look again!

Ex :

```
S=0;                    1
                        1
for(oloop=1; oloop <=N; oloop *=2)
    for(inloop=0; inloop<oloop; inloop++)
        cout << ---;
```

	N=1	N=4	N=8	N=16
0	1	2	3	4
1	1			

Bubble Sort

2 3 1 5 9 4

$C_1 (n-1)$

2 3 5 9

Finally BingO!

day/date

$$\begin{cases} 0 & n=0 \\ n+S(n-1) & n>0 \end{cases}$$

$Sn(0) = 0$

$Sn(1) = n^1 + S(0) = n+0 = n+0 = 1$

$Sn(2) = n^2 + S(1) = n+n+0 = 2n+0 = 4$

$Sn(3) = n^3 + S(2) = n+n+n+0 = 3n+0 = 9$

$\propto$ $= 16$

$\vdots$

$S(n) = n*n$

Asymptotic Time Complexity:

Approximate measure of time!

Logarithm:-

$\log_2(8) \Rightarrow$ Means how many times should 2 be multiplied to itself to obtain 8?

Ans $\Rightarrow$ (3)

```
for(i=1; i<=n; i=i*2)
    // Statement
```

After 1	$i=1$	2^0
After 2	$i=2$	2^1
After 3	$i=4$	2^2
After 4	$i=8$	2^3
After 5	$i=16$	2^4
$\vdots$	$\vdots$	$\vdots$
After k	$i=n$	2^{k-1}

Since $\boxed{n = 2^{k-1}}$

Finally BingO!

day/date

Now Solve this: $k-1 = \log_2(n)$ ★ $\boxed{k = 1 + \log_2(n)}$

```
for(i=n; i>=1; i=i/2)
  // Statement
```

After 1	$i = n/2^0$
After 2	$i = n/2^1$
After 3	$i = n/2^2$
After 4	$i = n/2^3$
⋮	⋮
After k	$i = n/2^{k-1} = 1$

$$\frac{n}{2^{k-1}} = 1 \Rightarrow n = 2^{k-1}$$

★ $\boxed{k = \log_2(n) + 1}$

★

```
int i,j,k, count=0.
for(i=n/2; i<=n; i++)        Outer
  for(j=1; j+n/2<=n; j++)    Inner
    for(k=1; k<=n; k=k*2)    Innermost
      count++;
```

Finally BingO!

day/date

Outer

Iter 1 $i = n/2 + 0$
Iter 2 $i = n/2 + 1$
Iter 3 $i = n/2 + 2$
Iter 4 $i = n/2 + 3$
⋮ ⋮
Iter k $i = n/2 + (k-1) = n$

→ Solve this

★ $\boxed{k = \frac{n}{2} + 1}$

Inner

Iter 1 $j = 1$
Iter 2 $j = 2$
Iter 3 $j = 3$
Iter 4 $j = 4$
⋮ ⋮
Iter k $j = K = n/2$

Since $J + \frac{n}{2} \leq n$

$\boxed{J = \leq n/2}$ ★

Innermost

Same as on Prev page $\quad k = n \rightarrow 2^{k-1}$

$\boxed{k = \log_2(n) + 1}$

So Time Comp

$$\left(\frac{n}{2}+1\right)\left(\frac{n}{2}\right)\left(\log_2(n)+1\right)$$

Now Q2

```
if (n ≤ 1) return;
int i,j;
for(i=1; i<=n; i++)
    for( j=1; j<=n; j++){
        cout....
        break;
    }
```

n times

break; ⇒ Watch out for this

Runs only once

* If Cases

Calculate if or else part which ever is worse

Finally BingO!

day/date

$i=1$	$i=2$	$i=3$	.....	$i=n$
Inner runs only once	Inner only once	only once		only once

Q3:-

```
for(i=1; i<=n/3; i++) → outer
    for(j=1; j<=n; j+=4) → Inner
```

const

Outer:-

$i=1$	Iter 1
$i=2$	Iter 2
$i=3$	Iter 3
⋮	⋮
$i=n/3$	Iter k

So $\boxed{k = n/3}$

Inner:- $1 + (o(4))$

Iter 1	$j=1$, ~~1 + 1(5)~~
Iter 2	$j=5$, ~~1 + 2(5)~~
Iter 3	$j=9$, ~~1 + 2(5)~~
Iter 4	$j=13$, ~~1 + 3(5)~~ → $1+3(4)$
⋮	⋮
Iter k	$j = 1+(k-1)4 = n$

$1 + 4k - 4 = n$

$4k = n+3$

$\boxed{k = \frac{n+3}{4}}$

Finally BingO!

day/date

$T(n) = 4T(n/2) + n^3$ ★ $T(1) = 1$

$T(n/2) = 4\left[4T(n/4) + (n/2)^3\right] + n^3$

$T(n/4) = 4\left[4\left[4T(n/8) + (n/4)^3\right] + (n/2)^3\right] + n^3$

$T(n/8) = 4\left[4\left[4\left[4T(n/16) + (n/8)^3\right] + (n/4)^3\right] + (n/2)^3\right] + n^3$

⋮

$T(n/4) = 4^3 T(n/2^3) + 4^2 (n/2^2)^3 + 4^1 (n/2^1)^3 + n^3$

$= 4^3 T(n/2^3) + \frac{n^3}{4} + \frac{n^3}{2} + \frac{n^3}{1}$

$T(k) = 4^k T(n/2^k) + n^3\left[\frac{1}{2^{k-1}} + \frac{1}{2^{k-2}} + \cdots + \frac{1}{2^0}\right]$

$= 4^{\log_2 n}(T(1)) + n^3\left[\frac{1}{2^{\log n - 1}} + \cdots\right]$

$= n^2 + n^3 \underbrace{\left[\sum_{i=0}^{\log n - 1} \frac{1}{2^i}\right]}_{\text{constant}}$

★ $O(n^3)$

★ Look at all the Recursive Cases

Finally BingO!

day/date

$T(n) = 2T(n/2) + n$ → additional cost. (2: no of subprobs; $n/2$: size of sub probs)

Tree Method

Level 0 — $T(n)$ — $2^0 \times n$

$T(1) = 1$ Given *

Level 1 — $T(n/2)$ $T(n/2)$ — $2^1 \times (n/2) = n$

Level 2 — $T(n/4)$ $T(n/4)$ $T(n/4)$ $T(n/4)$ — $2^2 \times (n/4) = n$

Level 3 — $T(n/8)$ ... — $2^3 \times (n/8) = n$

⋮

Base Case

Level k — $T(n/2^k)$ — $2^{\log_2 n} \times T(1) = n$

Value of k

↳ $\log_2(n)$

Total time = last attempt + All prev.

$$= n + \sum_{i=0}^{\log(n)-1} (n + n \dots + n)$$

$$= \boxed{n + n \log n} \; *$$

Finally BingO!

day/date

$T(n) = T(n/2) + n$

Level 01 $T(n/2) + 2^0 \times n/2$

Level 2 $T(n/4) + n/4$

level 3 $T(n/8) + n/8$

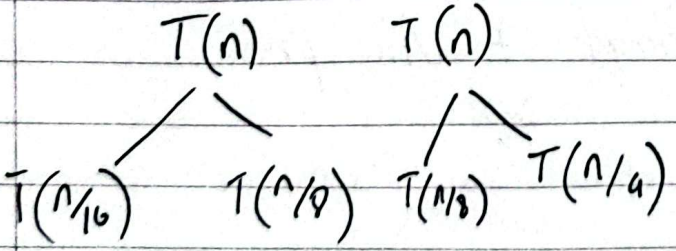
level 4 $T(n/16) + n/16$

⋮

Level k $T(n/2^k) + n \quad 2^{\log n} +$

Last + Prev.

$\boxed{T(n) = aT(n/b) + cn^k}$ Apply on this to get general



Finally BingO!

day/date

for (i - - i<n i*=2) — $\log_2 n$.

for (j - j<P ; j*=2) $\log_2 P$ => $\log_2(\log n)$

Finally BingO!

day/date

$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \dots < 2^n < n^n$

Big-oh (Upperbound)

$f(n) = O(g(n))$ iff $\exists$ +ive Constants. c and n_0.

$f(n) \le c * g(n)$ $\forall$ $n \ge n_0$

eg $f(x) = 2n+3$.

$2n+3 \le 10n$. So $c = 10$

This is our $g(n)$!

$n \ge 1$ * So $n_0 = 1$

for this case.

$f(n) = O(g(n)) = O(n)$

All these functions above (n) in Time are upper bound and below are lower bound.

$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 \dots$

lower Bound — Common Part / Average Bound — upper bound

Omega Notation :-

$f(n) \ge c * g(n)$ $\forall$ $n \ge n_0$. Lower Bound

$f(n) = 2n+3$

$2n+3 \ge 1n$ * $g(n)$ $\therefore f(n) = \Omega(n)$ ✓

c

$f(n) = \Omega(\log n)$ ✓

Bingo!

day/date

Theta Notation :-

$f(n) = \Theta(g(n))$ iff $\exists$ +ive c_1, c_2, n_0

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

$$1 \times n \leq 2n+3 \leq 5n$$

c_1 $\quad$ c_2

$\therefore f(n) = \Theta(n)$

Tighter bound.
We cannot use any other.

eg $n^2 \log n + n$

$$1 \times n^2 \log n \leq n^2 \log n + n \leq 10\, n^2 \log n$$

$O(n^2 \log n)$
$\Omega(n^2 \log n)$
$\Theta(n^2 \log n)$

* $n!$

$f(n) = n! = 1 \times 2 \times 3 \ldots n$

$$1 \times 1 \times 1 \times 1 \ldots 1 \leq 1 \times 2 \times 3 \ldots n \leq n \times n \times n \times n \ldots n$$

$$1 \leq n! \leq n^n$$

$O(n^n)$
$\Omega(1)$

* No theta

* We cannot find a place for it in the classes.
$(1 < \log n < \sqrt{n} < n \ldots)$

* $\log(n!)$

$$\log(1 \times 1 \ldots) \leq \log(1 \times 2 \times 3 \ldots n) \leq \log(n \times n \times n \ldots n)$$

$$1 \leq \log(n!) \leq \log n^n$$

$O(n \log n)$ $\quad$ $\Omega(1)$

Finally BingO!

day / date

* The most useful band is theta Big oh and Ω problem is that they can take many values for a single f(n). We don't know how close we are to one.

Properties

$f(n) \Rightarrow O(g(n))$ then $a * f(n) \Rightarrow O(g(n))$

$2n^2+5 \Rightarrow O(n^2)$ — Fxfn → $14n^2+35 \Rightarrow O(n^2)$.

Reflexive

$f(n) = n$ → $O(n)$.

$f(n)$ is $O(f(n))$

Big-oh ≥

every fcn is = to itself.

Transitive

$f(n) \to O(g(n))$ then $g(n) \to O(h(n))$

$f(n) = O(h(n))$

$f(n) = n$
$g(n) = n^2$
$h(n) = n^3$

Omega / Theta kay bugay bhi True.

Symmetric $f(n) \to \Theta(g(n))$ then $g(n) \to \Theta(f(n))$

Theta only

Bingo!

day/date

Comparison of functions :-

Value Sampling

n^2	n^3
2^2	2^3
3^2	3^3
4^2	4^3

So we can see which is bigger.

Log :-

(Apply on both Sides) :-

Log Formulas.

$\log(ab) = \log a + \log b$

$\log(\frac{a}{b}) = \log a - \log b$

$\log a^b = b \log a$.

$a^{\log_c b} = b^{\log_c a}$

$a^b = n \; ; \; b = \log_a n$

Now ex

$n^2 \log n \qquad n(\log n)^{10}$

Apply log

$\log(n^2 \log n)$	$\log(n(\log n)^{10})$
$\log n^2 + \log(\log n)$	$\log n + \log(\log n)^{10}$
$2\log n + \log(\log n)$	$\log n + 10 \log\log n$

Greater.

ex2 $3n^{\sqrt{n}} \qquad 2^{\sqrt{n}\log n}$

$3n^{\sqrt{n}}$	$2^{\log_2 n^{\sqrt{n}}}$
$3n^{\sqrt{n}}$	$(n^{\sqrt{n}})^{\log_2 2}$
$3n^{\sqrt{n}}$	$n^{\sqrt{n}}$

Now can see.

Finally BingO!

day/date

★ $\log\log n < \log n$

$2^{2n} = O(2^n)$ True or False ?

$4^n = O(2^n)$ Since $4^n > 2^n$, False

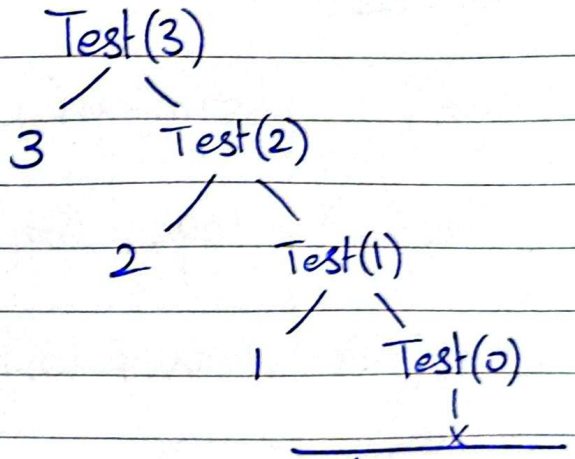
Divide & Conquer:-

★ Problem divided into Sub problems.

Combined solutions will give Ans.

Recurrence Relations:- & how to Solve them!

```
Void Test (n)
  if (n>0)
    cout--n
    Test (n-1)
```



If you take printing as major work.

n times prints
n+1 times calls fns

$T(n) = n$

If calling $T(n) = n+1$

Finally BingO!

day/date

Now finding the recurrence Relation!

Void Test (n) $(T(n)) \rightarrow ?$

if (n > 0)

Printing ; $\longrightarrow$ 1 unit time

Test(n-1); $\longrightarrow$ $T(n-1)$ Unit times

So Relation

$$\boxed{T(n) = T(n-1) + 1}$$

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

Iterative Method

$T(n-1) = T(n-2) + 1$

$T(n-2) = T(n-3) + 1$

$T(n) = [T(n-2) + 1] + 1$

$T(n) = T(n-2) + 2$

$T(n) = [T(n-3) + 1] + 2$

$T(n) = T(n-3) + 3$

⋮ k times

$T(n) = T(n-k) + k$

$T(n) = T(n-k) + k$

$n - k = 0$; $\boxed{n = k}$ last case

$T(n) = T(0) + n$

$\boxed{T(n) = 1 + n}$

$\boxed{n+1}$ → number of calls

Finally BingO!

day/date

Recurrence Relation

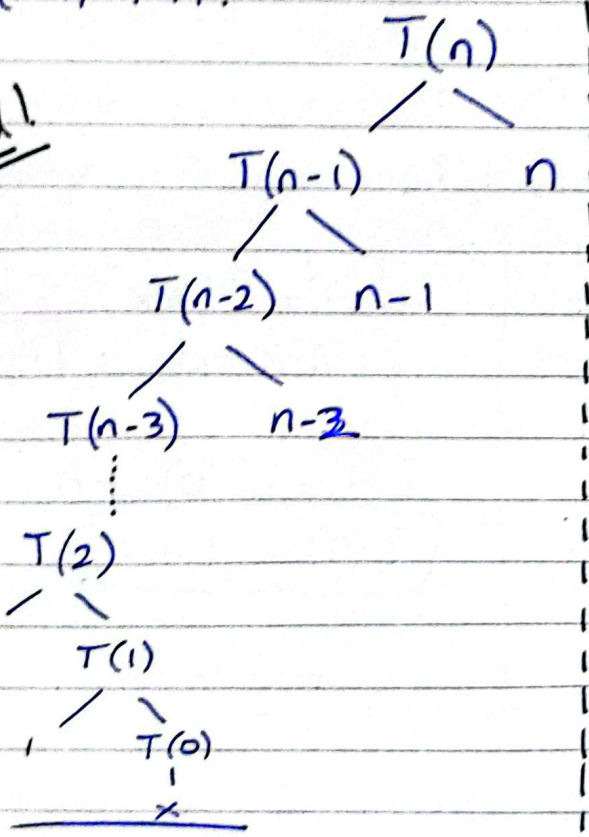
$T(n) = ?$

```
Test(n)
1- if(n>0)
n+1 ---- for (---- n times)
n   ------ count
Sum T(n-1)= Test(n-1);
```

$T(n) = T(n-1) + (2n+2)$ → Take Big oh of this. for Simplicity

$T(n) = T(n-1) + n.$

Tree Method!



So add all the Work being done on Each level.

$0+1+2+3 \ldots + n-1+n$

$= \frac{n(n+1)}{2}$

$T(n) = \frac{n(n+1)}{2}$

$O(n) = (n^2)$

Iterative

$T(n) = T(n-1) + n$

$T(n-1) = T(n-2) + (n-1)$

$T(n) = [T(n-2) + (n-1)] + n. = T(n-2) + (n-1) + n.$

$T(n) = [T(n-3) + (n-2)] + (n-1) + n$

$T(n) = T(n-3) + (n-2) + (n-1) + n.$

BingO!

day/date

$T(n) = T(n-k) + (n-(k-1)) + \dots$

Assume $n-k=0$; $n=k$.

$T(n) = T(0) + (n-n+1) + (n-n+2) \dots + (n-1) + n.$

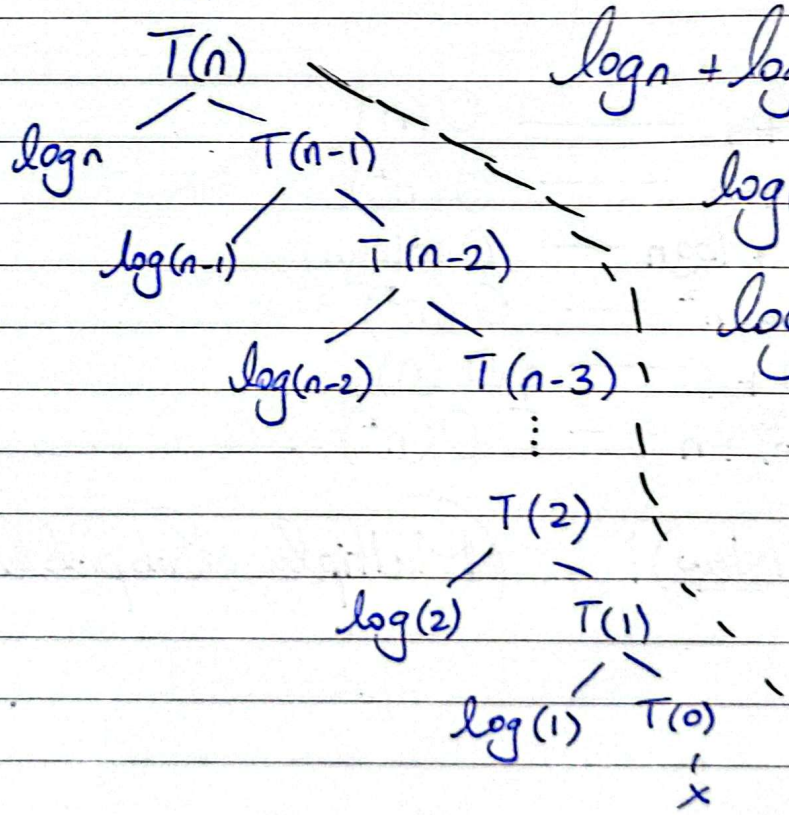
$T(n) = 1 + \frac{n(n+1)}{2}$. $\Theta(n^2)$

★ Another Example!

```
Test (n)
    if (n>0)
        for ( i=i+2 )
            cout
        Test(n-1);
```

$\log n$

$T(n) = T(n-1) + \log n ;$



$\log n + \log(n-1) + \dots \log(2) + \log 1$

$\log(n \times (n-1) \times (n-2) \times \dots 1)$

$\log(n!)$

$O(n \log n)$

Bing0!

day/date

Iteration Method!

$$T(n) = T(n-1) + \log n$$

$$T(n) = [T(n-2) + \log(n-1)] + \log n$$

$$T(n) = [T(n-3) + \log(n-2)] + \log(n-1) + \log n$$

$$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log n$$

$$\vdots$$

$$T(n) = T(n-k) + \log 1 + \log 2 + \dots + \log(n)$$

$$T(n) = T(0) + \log(n!) = 1 + \log(n!) \quad *$$

Now Pattern :-

$T(n) = T(n-1) + 1$ —— $O(n)$

$T(n) = T(n-1) + n$ —— $O(n^2)$

$T(n) = T(n-1) + \log n$ —— $O(n \log n)$

$T(n) = T(n-1) + n^2$ —— $O(n^3)$

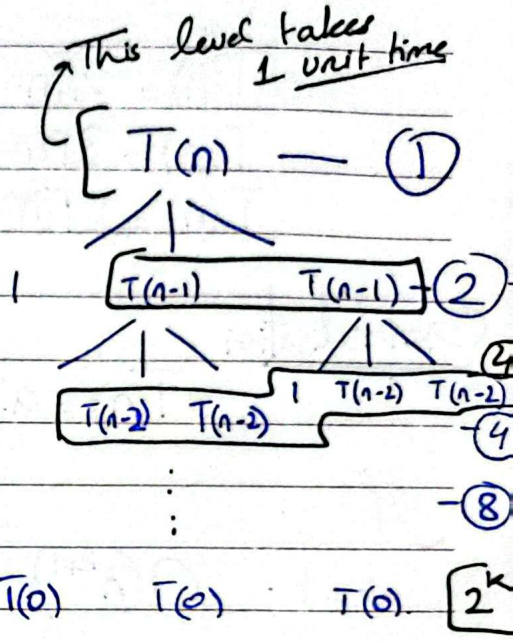
$T(n) = T(n-2) + 1$ —— $\frac{1}{2}O(n)$

$T(n) = T(n-100) + n$ —— $O(n^2)$

Now this Type (New)! (Multiple Subproblems)!

day/date

```
Algo(n)
    if (n>0)
        1 ——— print
        T(n-1) — Algo(n-1);
        T(n-1) — Algo(n-1);
```



$$T(n) = 2T(n-1) + 1;$$

$$1 + 2^1 + 2^2 + 2^3 + \dots 2^k$$

$$\frac{a(r^{k+1}-1)}{r-1} = 2^{k+1} - 1;$$

Assume $n-k=0$, $\underline{n=k}$

$$\boxed{2^{n+1}-1} \qquad O(2^n)!$$

Iterative Method!

$$T(n) = 2T(n-1) + 1$$

$$T(n) = 2\left[2T(n-2)+1\right] + 1$$

$$T(n) = 2^2 T(n-2) + 2 + 1$$

$$= 2^2\left[2T(n-3)+1\right] + 2 + 1$$

$$T(n) = 2^3 T(n-3) + 2^2 + 2 + 1$$

$$T(n) = 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \dots + 2^2 + 2 + 1.$$

$n-k=0$; $\boxed{n=k}$

$$T(n) = 2^n T(0) + \underbrace{1+2+2^2+\dots 2^{k-1}}$$

$$= 2^n + 2^n - 1$$

$$= 2^{n+1} - 1$$

Finally BingO!

day/date

Master theorm for Decreasing functions:-

$T(n) = 2T(n-1) + 1$ — $O(2^n)$

$T(n) = 3T(n-1) + 1$ — $O(3^n)$

$T(n) = 2T(n-1) + n$ — $O(n2^n)$

Generalisation

$T(n) = aT(n-b) + f(n)$. $a>0$ $b>0$ $f(n) = O(n^k)$ where $k \geq 0$

If

$a=1$; $O(n^{k+1})$ or $O(n * f(n))$

If

$a>1$; $O(f(n) * a^{n/b})$ or $O(n^k a^{n/b})$

* n/b Note

If

$a<1$; $O(n^k)$ or $O(f(n))$

finally BingO!

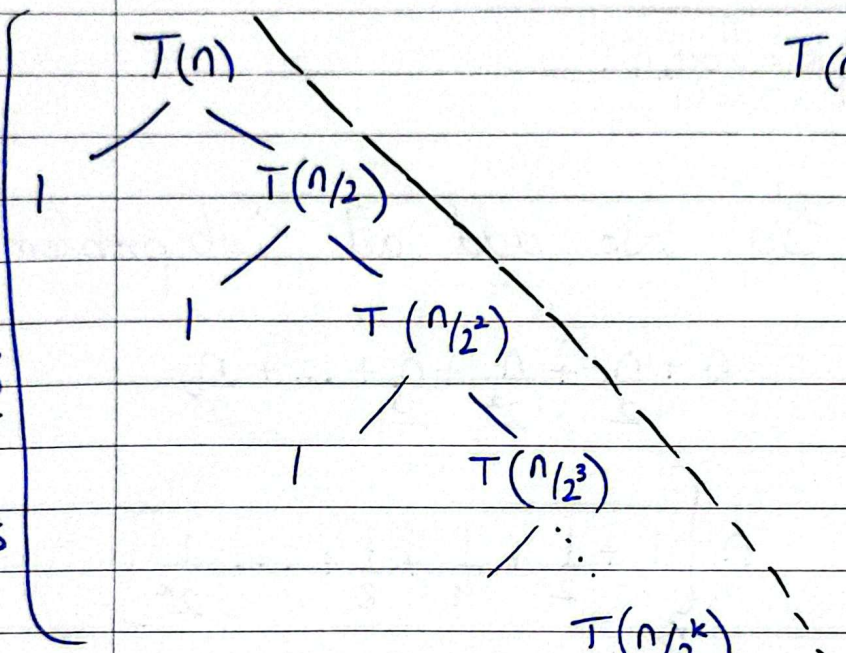
day/date

Dividing functions!

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$

```
Algo(n)
    if (n>1)
1 ——— cout ———
T(n/2) ——— Algo(n/2)
```

$T(n) = T(n/2) + 1$



$O(\log_2 n)$

k Steps

$\frac{n}{2^k} = 1$; $n = 2^k$.

$k = \log_2 n$

Some question Iterative method!

$T(n) = T(n/2) + 1$; $T(n/2) = T(n/2^2) + 1$

$T(n) = [T(n/2^2) + 1] + 1$

$T(n) = T(n/2^2) + 2$

$T(n) = T(n/2^3) + 3$

$T(n) = T(n/2^k) + k$

Assume $n/2^k = 1$ $k = \log n$

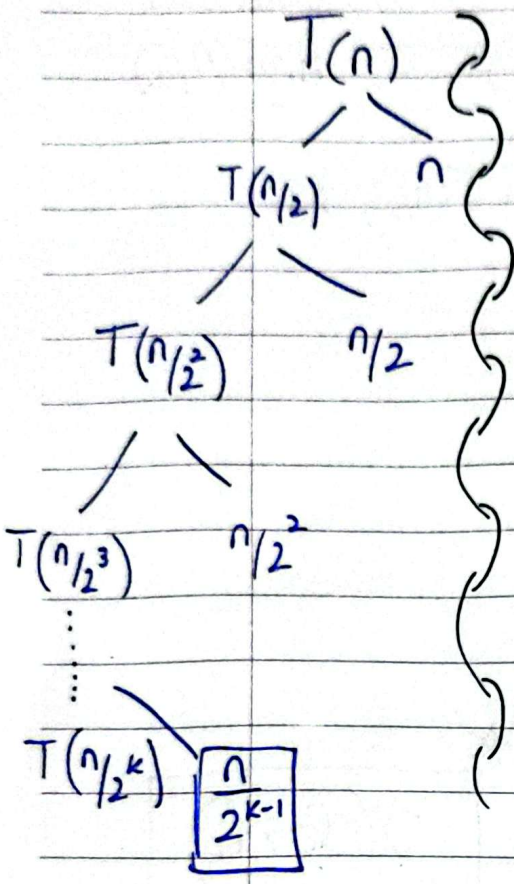
$T(n) = T(1) + \log(n)$.

$T(n) = 1 + \log n$.

Finally BingO!

day/date

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases}$$



$n/2^k = 1$

$k = \log_2 n$.

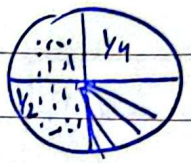
So We add all Sub problems.

$$n + \frac{n}{2} + \frac{n}{2^2} + \frac{n}{2^3} + \dots + \frac{n}{2^k}$$

$$n\left[1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \frac{1}{2^k}\right]$$

$$n \sum_{i=0}^{k} \frac{1}{2^i} = n \times 1 = n$$

$O(n)$



Iteratively :-

$T(n) = T(n/2) + n$

$T(n) = \left[T(n/2^2) + n/2\right] + n$

$T(n) = T(n/2^2) + \frac{n}{2} + n$

$T(n) = T(n/2^3) + \frac{n}{2^2} + \frac{n}{2^1} + \frac{n}{2^0}$

$\vdots$

$T(n) = T(n/2^k) + \frac{n}{2^{k-1}} + \frac{n}{2^{k-2}} + \dots \frac{n}{2^0}$

$k = \log n$.

$T(n) = T(1) + n\left[\frac{1}{2^{k-1}} + \frac{1}{2^{k-2}} + \dots\right]$

$= 1 + n[1+1]$

$= 1 + 2n$.

$O(n)$

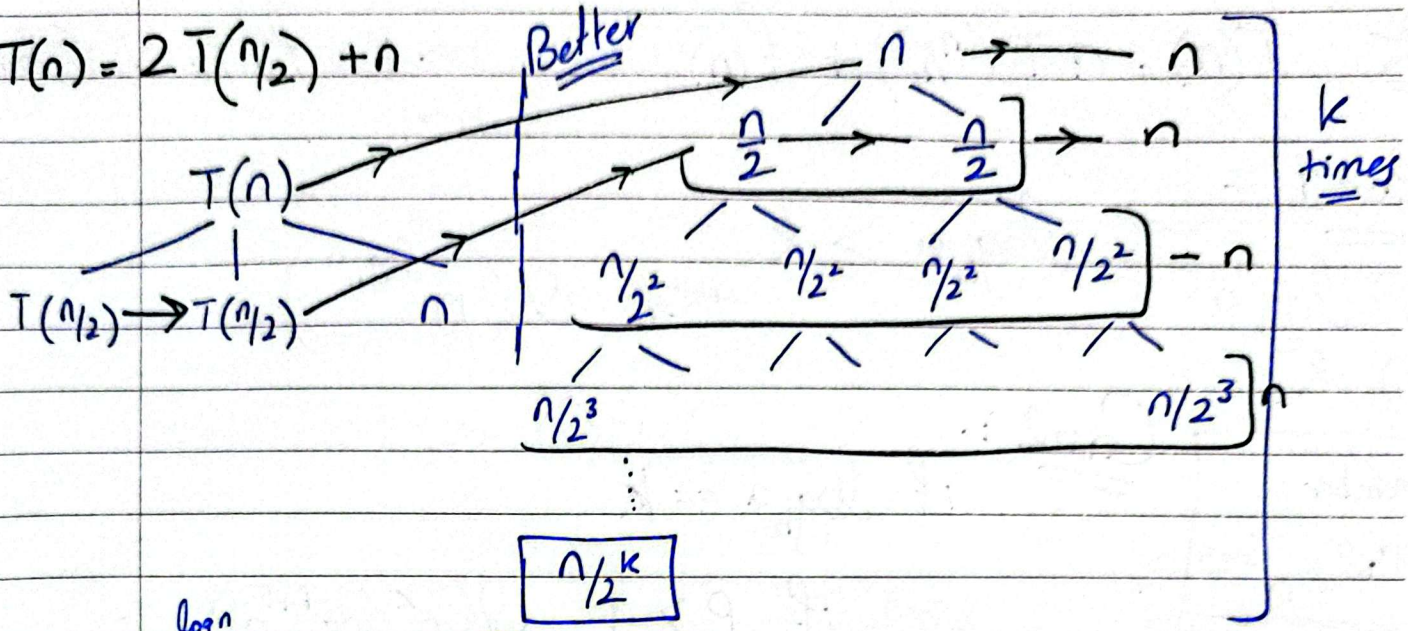
Finally BingO!

day/date

Rewrence

$T(n) = 2T(n/2) + n$

Better



$n/2^k$

$\sum_{i=0}^{\log n} n = n \log n$

Iterative

$T(n) = 2T(n/2) + n$

$T(n/2) = 2T(n/2^2) + \frac{n}{2}$

$T(n) = 2\left[2T(n/2^2) + n/2\right] + n$

$= 2^2 T(n/2^2) + 2\left(\frac{n}{2}\right) + n = 2^2 T(n/2^2) + n + n$

$= 2^k T(n/2^k) + kn$ ☆

$k = \log n$

$n = 2^k$

$T(n) = 2^k (T(1)) + kn$

$= (n \times 1) + n \log n$

☆ $O(n \log n)$

Finally BingO!

day/date

Masters Theorm (Dividing)

★ $T(n) = aT(n/b) + f(n)$ $\quad a \geq 1,\ b > 1$ $\quad f(n) = \Theta(n^k \log^p n)$

Case 1 -

① $\log_b a$

② k

$\log_b a > k$ then $\Theta(n^{\log_b a})$

Remember $a^{\log_b n} = n^{\log_b a}$

Case 2 :-

if $\log_b a == k$

if $p > -1$ $\quad \Theta(n^k \log^{p+1} n)$

$p == -1$ $\quad \Theta(n^k \log\log n)$

$p < -1$ $\quad \Theta(n^k)$

Case 3 :-

$\log_b a < k$ iff $p \geq 0$ $\quad \Theta(n^k \log^p n)$

$\Longleftrightarrow$ $\quad p < 0$ $\quad \Theta(n^k)$

$f(n) = \lceil n^k / \log n \rceil$ → Master fails

Explanation of Cases :-

① Weight increases Geometrically from root to leaves. So the last term is enough for $\Theta()$

② Weight is same on Each level So that's why we multiply by height of Tree which is $\log_b n$

③ Weight decreases Geometrically ↓ So $f(n)$ is the largest term.

Finally BingO!

day/date

Some Examples!

① $T(n) = 2T(n/2) + 1$. $a=2$, $b=2$, $f(n) = \Theta(1)$

$= \Theta(n^0 \log^0 n)$

$k=0$, $p=0$

$\log_a b = \log_2 2 = 1$ | $k=0$.

So Case 1

$\Theta(n^1) \Rightarrow \Theta(n)$

② $T(n) = 4T(n/2) + n$ Case 1

$\log_b a = 2$ | $k=1$, $p=0$ $\Theta(n^2)$

③ $T(n) = 2T(n/2) + n$

$\log_b a = \log_2 2 = 1$ | $k=1$ $p=0$

Case 2

$p > -1$ $\Theta(n \log n)$

④ $T(n) = 2T(n/2) + n^2 \log^2 n$

$\log_2 2 = 1$ | $k=2$

Case 3

$\Theta(n^2 \log^2 n)$

Limitations

* $T(n)$ Not montone $= \sin(n)$

Not Polynomial 2^n

a not constant $a=2n$

Finally BingO!

day/date

Master Theorem

$T(n) = aT(n/b) + f(n)$ $\quad a \geq 1, b > 1$

Case 1 If $f(n) = O(n^{\log_b a - \varepsilon})$ some $\varepsilon > 0$

then $T(n) = \Theta(n^{\log_b a})$

Ex

$T(n) = 9T(n/3) + n$

$\log_b a = \log_3 9 = \boxed{2}$

Now

$f(n) = n$] from Question ✓

does ↳ $f(n) = O(n^{2-\varepsilon})$ * Can I enter some value of ε to satisfy this?

If $\boxed{\varepsilon = 1}$ $f(n) = O(n)$ ↳ True ✓

So $T(n) = O(n^2)$

Case 2 :- If $f(n) = \overset{\text{bounded}}{\Theta}(n^{\log_b a})$

$T(n) = \Theta(n^{\log_b a} \times \log n)$ then

Ex

$= T\left(\frac{2n}{3}\right) + 1$ $\quad a = 1$; $b = 3/2$

$f(n) = 1$; $n^{\log_{3/2} 1} = n^0 = 1$.

$\boxed{f(n) = \Theta(1)}$ So $T(n) = (n^0 \times \log n)$

$\boxed{T(n) = \log n}$

Bingo!

day/date

Case 3 if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ $\varepsilon > 0$

AND

$a f(\frac{n}{b}) \leq c \cdot f(n)$; $c < 1$ $\forall n$

ex $T(n) = 3T(n/4) + n\log n$ $a=3$; $b=4$

$\log_4 3 = 0.793$ $f(n) = n\log n$

$f(n) = n\log n$; $f(n) = \Omega(n^{0.793+\varepsilon})$ some $\varepsilon > 0$

Take $\varepsilon = 0.2$ → $f(n) = \Omega(n^{1})$ ✓ (True)

* Now Check 2nd Condition:-

$a f(n/b) \leq c f(n)$ Replace

$3\left[\frac{n}{4}\log\frac{n}{4}\right] \leq c \cdot n\log n$ $\forall n$; $c < 1$

let $c = \frac{3}{4}$

$\frac{3}{4} n \log\frac{n}{4} \leq \frac{3}{4} n \log n$

* So there exists a value of c for which whatever n the relationship is satisfied!

So $T(n) = \Theta(n\log n)$!

Finally BingO!

day/date

Root function

$$T(n) = \begin{cases} 1 & n = 2 \\ T(\sqrt{n}) + 1 & n > 2 \end{cases}$$

$T(n) = T(\sqrt{n}) + 1$

$T(n) = T(n^{\frac{1}{2}}) + 1$

$T(n) = T(n^{\frac{1}{2^2}}) + 2$

$T(n) = T(n^{\frac{1}{2^3}}) + 3$

$\vdots$

$T(n) = T(n^{\frac{1}{2^k}}) + k$

Assume $n = 2^m$

$T(2^m) = T(2^{\frac{m}{2^k}}) + k$

$T(2^{m/2^k}) = T(2^1)$

$\frac{m}{2^k} = 1$

$k = \log_2 m$

Since $n = 2^m$

$m = \log_2 n$

$k = (\log_2(\log_2 n))$

So

$T(n) = O(\log \log (n))$

Binary Search!

```
RbinSearch(l, h, key){
if (l == h)
    if(A[low] == key)
        return l;
    else
        return -1;
else
    mid = (l+h)/2;
```

Finally BingO!

day/date

```
if (A[mid] == key)
        return mid;
if (key < A[mid])
        return RBinS(l, mid-1, key);
else
```

$$T(n) = \begin{cases} 1 & n=1 \\ T(n/2) + 1 & n>1 \end{cases}$$

Heap & Heap Sort!

Node i → Left Child $= 2*i$ RC $= 2*i+1$.

Parent $= \lfloor \frac{i}{2} \rfloor$

Full BT $(2^{h+1}-1)$ → Nodes Max! → All leaf same lvl

Complete BT ⟹ Start filling from left (No dashes)

Insert Heap

* Last free space in Array!
* Percolate up.

eg index = 8

compare with $\frac{8}{2} = 4$

$\frac{4}{2} = 2$

$\frac{2}{2} = 1$

Swap Swap

Delete Heap

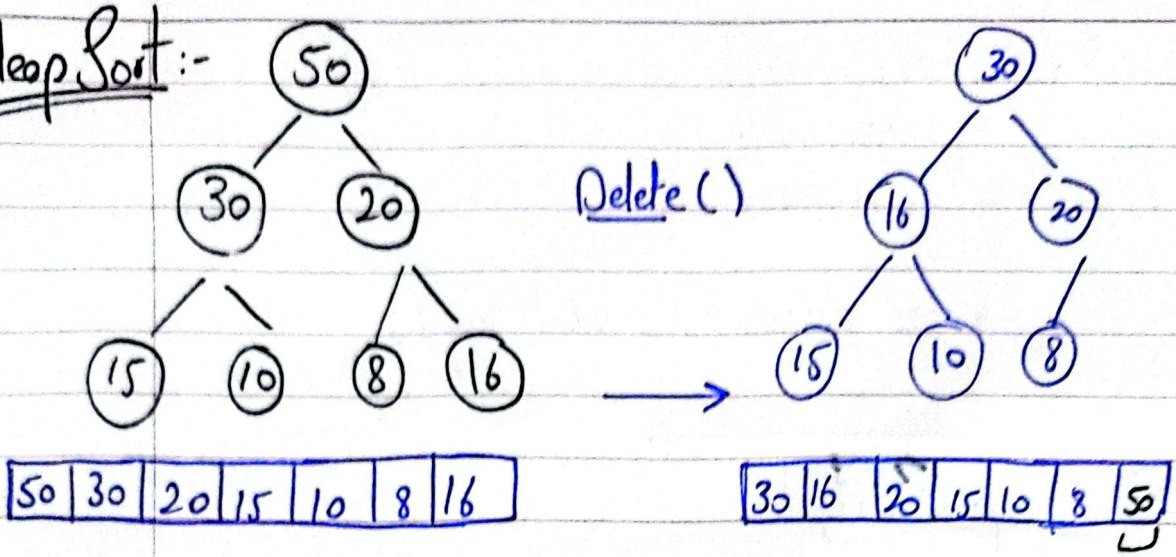
(Root deleted)

* Last element in Array or C.B.T takes its place.
* Choose the greater child & swap and keep swapping!

Finally BingO!

day/date

Heap Sort :-



50	30	20	15	10	8	16

30	16	20	15	10	8	50

empty space here!

* We removed the Max
Max was 50.
Now place 50 at the end!

⟹ Keep removing Max and placing at end!

Heapsort 1) Add all elements to and create Heap!
2) Delete all elements from Heap!

* T(n) for first step! → $\boxed{n \log n}$ n elements each moves Max logn

T(n) for deletion → nlogn.

O(nlogn)!

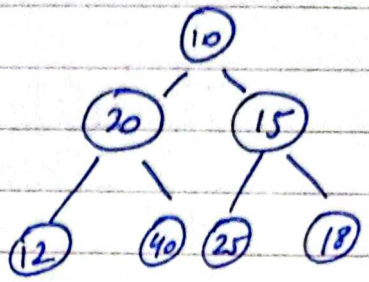
Finally BingO!

day/date

Heapify :-

Turn an array of numbers to a heap!

10	20	15	12	40	25	18



Start from $\lfloor n/2 \rfloor$ and move backwords in array!

$\lfloor 7/2 \rfloor = 3$

3
2
1

→ Percolate down on all of them

* Now Move 15 (percolate down)

Heapify

O(n) How?

Each call O(logn)
No of times (O(n)) nlogn?

Merge Sort :-

Merge:

A	B	C
i → 2	j → 5	2 ← k
8	9	5
15	12	8
18	17	9
m	n	12
		15
		17
		18

If A[i] < A[j]
copy A[i]
i++ ; k++

If one of lists finished then copy remaining elements.

*

O(m+n)

finally BingO!

day/date

DAC

```
Merge (A,B,m,n)
while (i<=m && j<=n){
    if (A[i] < B[j]){
        C[k++] = A[i++];
    else
        C[k++] = B[j++];
}
for (; i<=m; i++)
    C[k++] = A[i];
for (; j<=n; j++)
    C[k++] = A[j];
```

```
MS(l, h)
if n == 1
    return the ele
int a* = MS( )
int b* = MS( )
return Merge(a,b);
```

mid = (l+h)/2

merge (l, mid, h)

$O(n \log n)$ ★

$2T(n/2) + n$

Matrix Multiplication:-

$$A\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B\begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = C\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$m \times \boxed{n \quad n} \times m$

must

$$C_{ij} = \sum_{k=0}^{n} A_{ik} * B_{kj} \quad ★$$

```
for (i — n)
    for (j — n)
        C[i,j] = 0;
        for (k — n)
            C[i,j] += A[i,k] * B[k,j];
```

$O(n^3)$!

Finally BingO!

day/date

2×2

$C_{11} = a_{11} * b_{11} + a_{12} * b_{21}$

$C_{12} = a_{11} * b_{12} + a_{12} * b_{22}$

$C_{21} = a_{21} * b_{11} + a_{22} * b_{21}$

$C_{22} = a_{21} * b_{12} + a_{22} * b_{22}$

We assume it is powers of 2 (Size)

1×1

$[a_{11}]$ $[b_{11}]$

$C_{11} = a_{11} \times b_{11}$

$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$ A_{11} A_{12} A_{21} A_{22} — $\frac{4}{2} \times \frac{4}{2}$

$B = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}$ B_{11} B_{12} B_{21} B_{22} — $\frac{4}{2} \times \frac{4}{2}$ → divide by 2

★ Now treat A_{11} as 1 element.

Algo ( )

if ($n \leq 2$)

4 formulas;

else

mid $n/2$

$MM(A_{11}, B_{11}, n/2) + MM(A_{12}, B_{21}, n/2)$ ← Matrix addition

$MM(A_{21}, B_{12}, n/2) + MM(\quad)$

$$T(n) = \begin{cases} 4 & n \leq 2 \\ 8T(n/2) + n^2 & n > 2 \end{cases}$$

$\Theta(n^3)$

8 Muls

↓

7 Muls

P =

Q =

R =

S =

T =

U =

V =

7

$7T(n/2) + n^2$

Finally BingO!

day/date

Quick Sort!

x => pivot

★

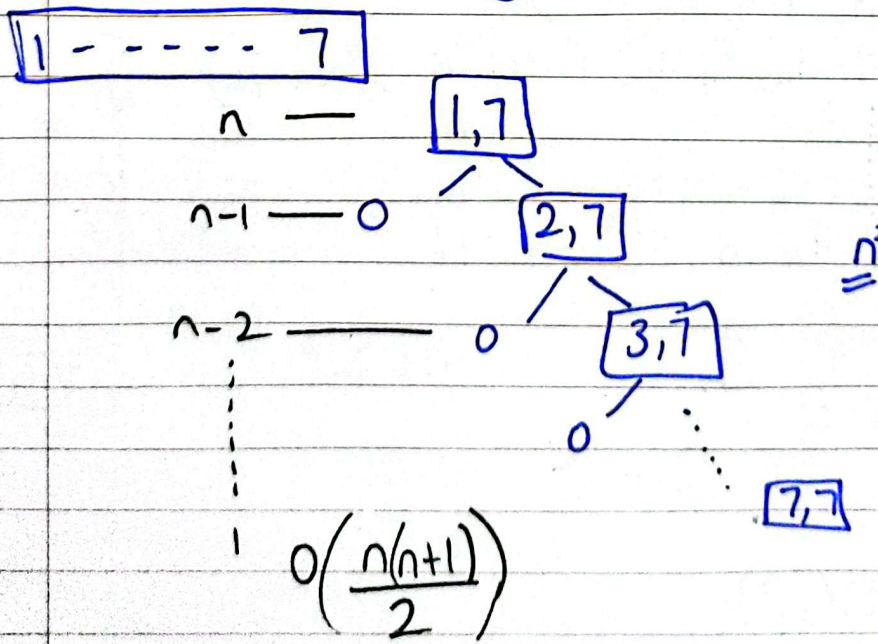
```
QS(A, l, h)                          O(n) Partition
i = l-1; x = A[h];
for ( j = l → h-1)
    if ( A[j] <= x)
        i++;
        swap A[i], A[j]
swap (A[i+1], A[h])
```

```
Main QS ( l, h)
    if (l < h)
        j = partition(l,h)
        QS (l, j) ;
        QS (j+1, h);
```

Partitioning at each level takes O(n)

Middle Partition Best (O(nlogn)) [Median]!

QS → Worst → already Sorted. $O(n^2)$!



Finally BingO!

day/date

// If we select the middle element always then even if the list is already sorted we get O(n log n)

Median of the 3 Rule :- l m R choose Median

Easy 2 avoid worst-case. | Better Space Complexity than MS

Counting Sort!

You create another array of size Max element in original array!

K → Maximum ele in orig array!

```
for i-1 — K
    C[i] = 0
for j → 1 → length[A]
    C[A[j]] = C[A[j]] + 1 .
for i → 2 → K
    C[i] = C[i] + C[i-1] ;
for length[A] downto 1
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[i]] - 1 ;
```

A original
C Sum Array
B Result.

Current Count

Prefix Sum Calculation!

Prefix Sum Calculation!

Placement!

$O(k+n)$

Finally BingO!

day/date

Bucket Sort :- [0,1)

N = 10

i^{th} bucket has $[i/10, (i+1)/10]$

Insert into array.

On each linked list perform Insertion sort!

Combine all list. (already in Seq)

Radix Sort !

Perform a Stable sort (Counting) on each digit of the number.

720

↑ 3rd ↑ 1st

d = no of digits.

$O(nd+nk)$

$2T(n/2)+O(1)$

Min-Max

If $|S| = 2$

$S = \{a,b\}$

return $\langle Max(a,b), Min(a,b) \rangle$

else

$\langle Max1, Min1 \rangle = MAXMIN(S_1)$ → each half size

$\langle Max2, Min2 \rangle = MAXMIN(S_2)$

return $\langle Max(Max1, Max2), Min(Min1, Min2) \rangle$

Finally BingO!

day/date

Rabin Karp Algorithm

★ Pattern : abc

Think
↳ we have ASCII | We can make pattern into 1 single value.

★ This reduces Time of matching!

Text Size = n
Pattern Size = m

a a a a a b
1+1+1 1+1+2

↙ No Match

→ Match
Now check each character

★ This can be our own hash fn

a - 1
b - 2
c - 3
⋮
j - 10

we can define it like this | eg

Key Idea of Rolling hash fn :-↘

a a a a a a ⇒ a a a a a a
1+1+1 □ 1+1 +1

*I removed this and added this
So less computing cost!

★ Calculating the hash value for next pattern calculated in Constant time!

★ If no hash matches $O(n-m+1)$ Avg

★ Spurious hits ⇒ Hash code matches but not patterns (Waste)

Worst $O(nm)$

↳ ★ To Avoid Strong hash Function!

Finally BingO!

day/date

Strong hash ftn & Rabin Karp:- *

look → d b a

$4\times10^2 + 2\times10^1 + 1\times10^0$

421 *

Since we have a-j = 10
We took 10 as base.
All english chars Take $26^2, 26^1 \ldots$

* Rolling hash ftn → Horner's Rule!

$$\left[\left[\left(3\times10^2 + 3\times10^1 + 1\times10^0\right) - 3\times10^2\right]\times10\right] + \underset{\text{next code}}{___}$$

Rabin fingerprint ftn! ← $O(n-m+1)$

because ↓ spurious

Still worst $O(nm)$ → Spurious still possible.

Overflow

If you believe it will overflow, not fit in variable. You can take mode → ↑ Spurious.

Σ → all possible characters
Σ^* → all finite length strings from

$w \sqsubset x$ prefix
$w \sqsupset x$ suffix

ε → empty string → Both Prefix / Suffix of all.

* Choice of modulus

Size of modulus should be greater than $|\Sigma|$ to encompass all characters and avoid trivial collisions.

Finally BingO!

day/date

KMP (Algo) & Finite State Automata!

→ Study the pattern and then make decisions!

Naive
O(mn)

Need to know — abcdabc

Prefix: a, ab, abc, abcd...

Suffix: c, bc, abc, dabc, ...

☆ We basically check if beginning part of a patten appears in the end as well.

Generate π table

LPS → Longest Prefix same as Suffix.

P1: abcdabeabf
0 0 0 0 1 2 0 1 2 0

P2: abcdeabfabc
0 0 0 0 0 1 2 0 1 2 3

P3: aabcadaabe
0 1 0 0 1 0 1 2 3 0

P4: aaaabaacd
0 1 2 3 0 1 2 0 0

String: a b a b c a b c a b a b a b d

0	1	2	3	4	5
a	b	a	b	d	
0	0	1	2	0	

Start j from here

☆ Compare i with j+1
→ Match inc both.
☆ Mismatch then look at j in patt table.
eg if j here move it to 2
so j at index 2

☆ Mismatch and j at index 0 then move i.

Finally BingO!

day/date

* If j reaches the last index in π table pattern found.

↳ then move $j \leftarrow \pi(j)$

```
for i ← 1 to n                    ┐
    do while q>0 and P[q+1] ≠ T(i)│
        do q ← π(q)               │
    if P[q+1] = T[i]              │
        then q ← q+1              │
    if q=m                        │
        found                     │
        q ← π[q]                  ┘
```

thats it

Compute Prefix :-

```
π[1] = 0
k ← 0
for q ← 2 to m
    do while k>0 and P[k+1] ≠ P[q]
        do k ← π[k]
    if P[k+1] = P[q]
        k ← k+1
    π[q] ← k
```

eg) aabcadaabe

π	0	1	2	3	4	5	6	7	8
	a	a	b	c	a	d	a	a	b
	0	1	0	0	1	0	1	2	3

Finally BingO!

day/date

FS

Transition function

P = AAB$#C$#$#

10 chars

10 states.

10 + 1 = 11

State

State		A	B	C	$	#
q_0	A	1	0	0	0	0
q_1	AA	2 (AA)	0 (AB)	0 (AC)	0 (A$)	0 (A#)
q_2	AAB	2 (AAA)	3 (AAB)	0 (AAC)	0 (AA$)	0 (AA#)
q_3	AAB$	1 (AABA)	0 (AABB)	0	4 (AAB$)	0 (AAB#)
q_4	AAB$#					
q_5	:					
q_6	:					
q_7						
q_8						
q_9						
q_{10}						

Each step take this and calculate suffix of this.
Calculate Prefix of this.
Match Suffix & Prefix. Note the Max length.

```
for i ← 1 to n
    do q ← δ(q, T[i])
    if q = m
        found
```

→ Stage

Function

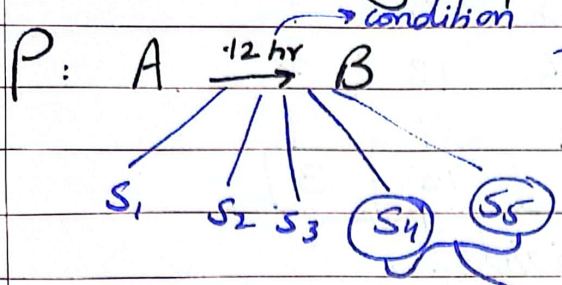
```
for q ← 0 to m
    for each character a ∈ Σ
        do k ← min(m+1, q+2)
            repeat k ← k - 1
            until P_k ⊐ P_q a
        δ(q, a) ← k
```

Finally BingO!

day/date

Greedy :-

Usually Optimisation Problems.



Since condition of 12 hr, I can't walk so train or flight.

These are Feasible Solutions! They satisfy our constraints.

So Travel in Min Cost.
↳ Minimisation problem.

So Train ⇒ optimal solution because (Quick + Min cost.)

3 Strategies to Solve Optimisation probs.

1. Greedy
2. Dynamic
3. Branch & bound

Should solve in stages.

```
for i ← 1 to n
    x = select(a);   // a: array
    if feasible(x)
        sol += x;
```

Greedy ⇒ locally optimal choice
↳ Not always best / but sometimes ✓

Greedy Activity selector :-

Sort all activities by finish time
O(n log n)

Then in O(n) select comptabile time activities!

Finally BingO!

day/date

Knapsack Problem :-

→ Fractional

n = 7

n = 15

objects	1	2	3	4	5	6	7
Profits	10	5	15	7	6	18	3
Weight	2	3	5	7	1	4	1

5	1·3	3	1	6	4·5	3

Calculate Profit by weights!

$\frac{P}{W} = 5$

So we can see that we should choose ob 5 then ob 1 then ob 6

15 - 1 = 14

14 - 2 = 12

12 - 4 = 8

8 - 5 = 3

3 - 1 = 2

2 - 2 = 0

$x \left(1 \; \frac{2}{3} \; 3 \; 5 \; 6 \; 7\right)$

ob → 4 → not included

Connected :-

If there is a path from one every vertex to every other vertex

Strongly :- If it is a directed + Connected graph

Weakly :- if the underlying undirected graph of a directed graph is connected!

Finally BingO!

day/date

BFS & DFS:

level order — Pre order

BFS:- Use a queue!

* Pick a vertex from Queue dequeue.
* Mark a visited, Mark all adjacent as visited and add them to queue!

→ if vertex adjacent is already true, then don't count and don't add to queue

Additionally, we can have an array for distance from start!

→ set distance = curr_dist. of all adjacent nodes.
then at end distance = curdistance + 1

DFS:- Use Stack!

* Start ko print (initial setup)
→ find adjacent (if not True yet call DFS(new))
→ add current to stack!

1st → Mark current as visited

Finally BingO!

day/date

MST (Min Spanning Tree)

Find Tree that connects all vertices at lowest cost.
→ No cycles.

→ Spanning Tree has $|V| = n = 6$ eg

→ $|V| - 1 = 5$ edges will be there.
→ no of edges

How many d/f Spanning Trees possible? ⇒ ${}^{6}C_{5} = \boxed{6}$
→ 1 less.

$$\boxed{{}^{|E|}C_{|V|-1} - \text{no of cycles}} \star$$

Weighted Graph :-

Prim's :- 1) Select Min edge!

2) Select a connected [Min edge] search
→ current key

Greedy Method!

→ Insert to PQ (source only)
Give all key ∞
Root key = 0
While (Q not Empty)
u = ExtractMin(PQ)
for Each neighbor
if (not in MST && key[n] > weight(u,n)
key[n] = weight[u,n]
insert n to PQ.

→ check it forms no cycle

Will have to maintain a parent for final answer

Finally BingO!

day/date

Another method of prims

Add all vertices in PQ at start.
assign all ∞ weights.
Assign root weight = 0.
Maintain parents!

★ for neighbours if (v ∈ Q and w(u,v) < key[v]
update key
update parent.

Extract min will
use the key array we are maintaining that had ∞ in beginning to decide which to remove.

Heap → $O(E \log V)$

Single Source Shortest path (Algo)

Dijkstra's (Greedy Method) ★ Consider 1 at a time!

if (d[u] + c(u,v) < d[v])

d[v] = d[u] + c(u,v);

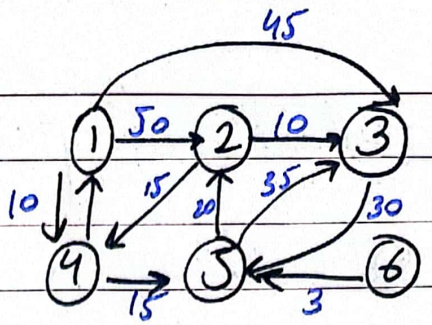
★ from the unvisited nodes, selected the shortest key one

$O(n \times n)$ ⇒ Worst case!

$O(|V|^2)$

Finally BingO!

day/date Start 1



Selected	2	3	4	5	6
4	50	45	(10)	∞	∞
5	50	45	(10)	(25)	∞
2	(45)	45	(10)	(25)	∞
3	(45)	(45)	(10)	(25)	∞
6	(45)	(45)	(10)	(25)	(∞)

```
for each v ∈ V
    d[v] = ∞

d[s] = 0; S = ∅; Q = V;

while (Q ≠ ∅)
    u = ExtractMin(Q);
    S = S ∪ {u};
    for each v ∈ u->Adj[]
        if (d[v] > d[u] + c(u,v)) {
            d[v] = d[u] + c(u,v); }
```

Relaxation step

$O(E \log V)$ → heap

day/date

Huffman Coding - Greedy :-

BCCABBDDAECCBBAEDDCC 20 len

★ Can we use our own codes instead of ASCII ?

Character	freq	code
A	3	000
B	5	001
C	6	010
D	4	011
E	2	100

ASCII - 8 bit
A 65 01000001
B 66 01000010

8 × 20 = 160 bits for message.

2 bits → 4 combinations
3 bits → 8 comb possible.

ASCII has 8 bits since we need to represent so many But our message has 5 chars only.

3 × 20 = 60 bits.
Size is reduced!

How will the reciever know the codes?
★ Need Table also.
↳ sending table 5 chars. their ascii 8 bits.

60 + 55 = 115 bits ★

5 × 8 = 40 bits.
5 × 3 = 15 bits. (codes)

Finally BingO!

day/date

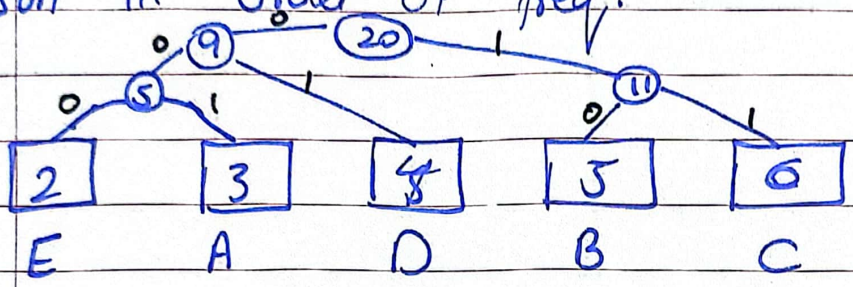
Variable Sized codes

chor	freq	code	
A	3	001	3×3 = 9
B	5	10	5×2 = 10
C	6	11	6×2 = 12
D	4	01	4×2 = 8
E	2	000	2×3 = 6

45 bits.

Need to send Table.

First sort in order of freq?



Mark left hand edges as 0

Mark right hand edges as 1

Take 2 smallest & merge

8×5 = 40 bits.

12 bits code. 52 bits table.

45+52 = 97 bits.

* $\boxed{\Sigma d_i * f_i}$ ⇒ Size of message from tree.

How to decode?

Easy Use table

BCCABB

10.11.11.001.10.10

Read a bit check if chor formed from Tab

Read a bit again and check.

day/date

Huffman

```
n ← |C|
Q ← C        (characters in PQ)
for  i ← 1 to n-1
     do z ← Allocate node
        x ← left[z] ← ExtractMin(Q)
        y ← right[z] ← "
        f[z] ← f[x] + f[y]
        INSERT (Q, z);
```

Heaps :-

3 Ops MaxHeapify() heapsort()

Build-Max-heap()

```
l ← left(i)
r ← righ(i)
if l ≤ n AND A[l] > A[i]
   then largest ← l
   else largest ← i
if r ≤ n AND A[r] > A[largest]
   then largest ← r
if largest ≠ i
        exchange A[i] ↔ A[Largest]
        MAX-heapify (__).
```

O(h)

O(log n)

Finally BingO!

day/date

BMH

for $i \leftarrow \lfloor n/2 \rfloor$ to 1 $O(n \log n)$?

do MAX-heapify ( )

No → Actually it is $O(n)$

Heap Sort :-

BMH(A)
for $i \leftarrow$ len[A] to 2
do exchange $A[1] \leftrightarrow A[i]$
MAX-Heapify ( ) ;

$O(n \log n)$

Finally

day/date

BMH

for $i \leftarrow \lfloor n/2 \rfloor$ to 1 — $O(n \log n)$?

do MAX-heapify ( )

No → Actually it is $O(n)$

Heap Sort:-

BMH(A)
for $i \leftarrow$ len[A] to 2
do exchange $A[1] \leftrightarrow A[i]$
MAX-Heapify ( ) ;

$O(n \log n)$

Build heap → $O(n \log n)$

Insertion → Deletion → $O(\log n)$

Min/Max $O(1)$

Specific element in a heap $O(n)$!

Prims Algo ⇒ $O(V^2)$ Adj Matrix!
$O(E \log V)$ with Adj list!

Dijkstra ⇒ $O(V^2)$ but Min PQ $O(V + E \log V)$!

Finally BingO!

day/date

Time of BFS & DFS $\Rightarrow$ $O(V+E)$.

* Sum of Vertix degrees is $2E$

All spanning Trees have exactly $N-1$ edges. ← MST (DFS Tree)

Prefix Constraint:-

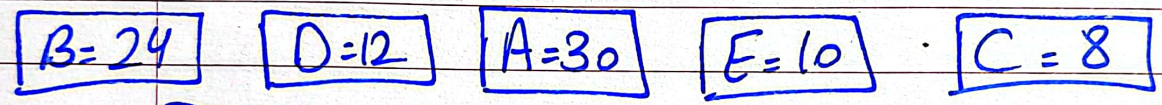
No codeword is a prefix of another.
Otherwise Ambiguity in determining message.

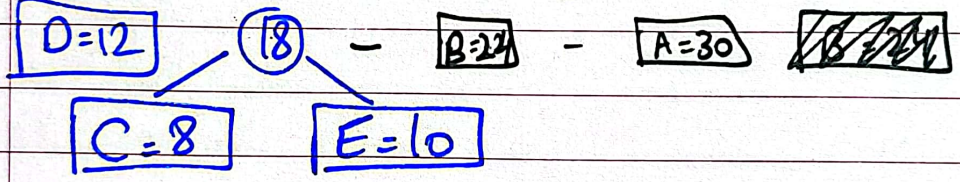
→ Maintained by code Tree!

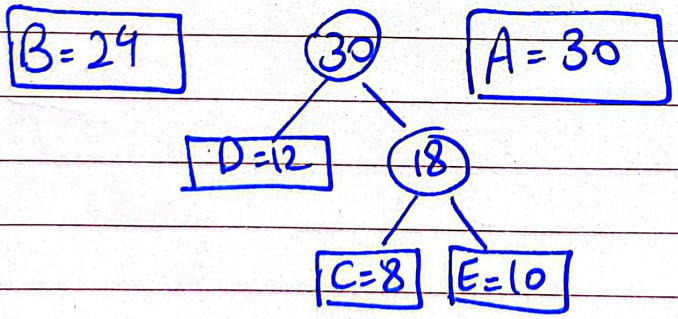
Constructing a huffman Code Tree!

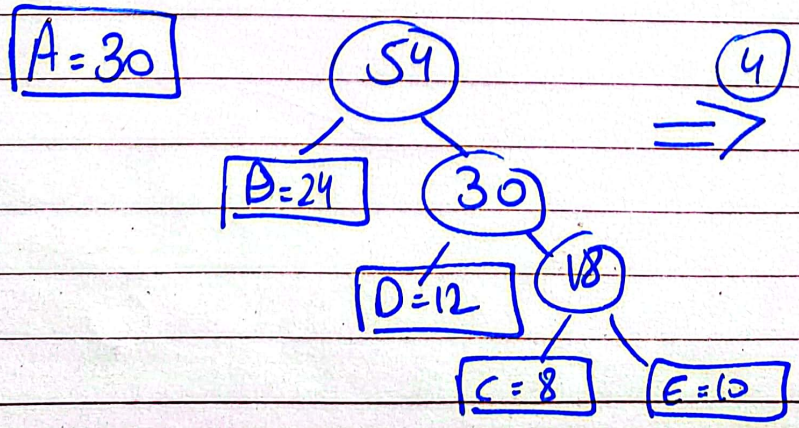
has |C| leav
|C|-1 → Internal nod

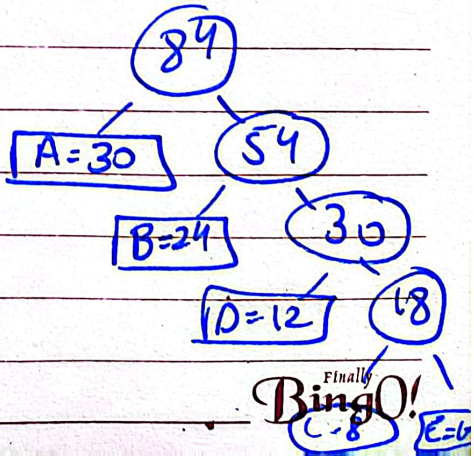
Always keep it sorted at each step.



① 

② 

③ 

④ ⇒ 

Finally BingO!

day/date

* After calculating the tree, you can find the codes for each letter. Then multiply freq by size of code and add to get min bits to store message.

To send msg, need additional bits for Table

Finally BingO!

day/date

Dynamic Programming!

4|5 3|1 2
45|3 12
453|12
4531|2

* Repetition in Sub problems!
* Optimisation Problems ⇒ Several solutions but we need best!

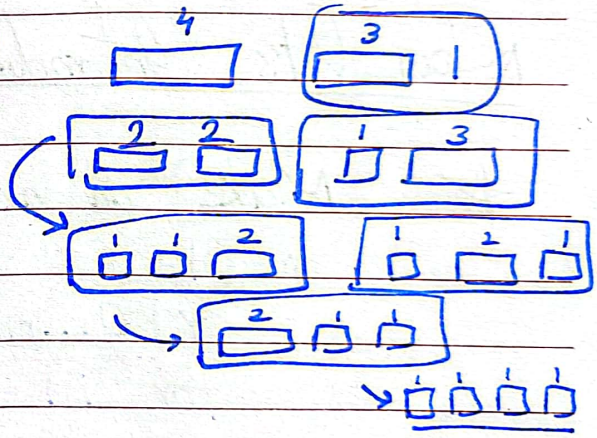
⇒ Rod Cutting Problem:-

Table that contains price of all d/f size of rod!

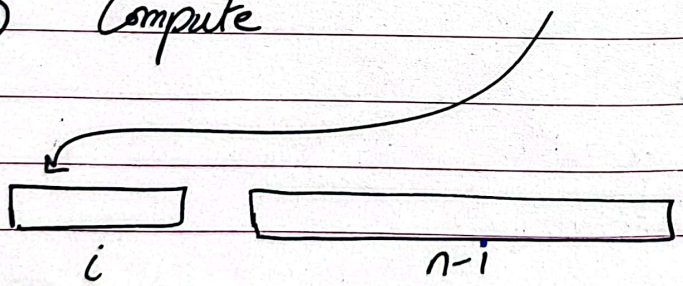
Length	1	2	3	4	5	6	..
price	1	5	8	9	10	17	

Number of ways to cut a rod of size n.

2^{n-1} If $n=4$, $2^3 = 8$



1) Characterise
2) Recursively define
3) Compute



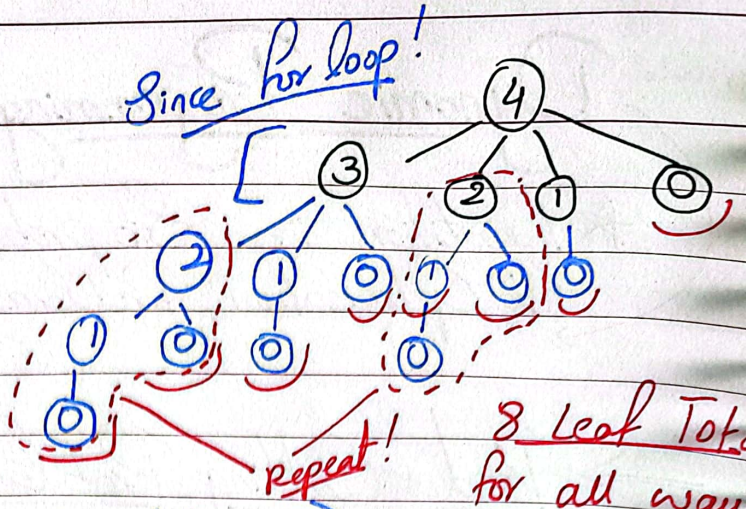
size:

$$r_n = \max \begin{cases} p_1 + r_{n-1} \longrightarrow \text{cut at pos} = 1, \text{ rest recursively} \\ p_2 + r_{n-2} \longrightarrow \text{cut at pos} = 2, \text{ rest recursively} \\ p_3 + r_{n-3} \longrightarrow \text{cut at pos} = 3, \text{ "} \\ \vdots \\ p_n + r_0 \end{cases}$$

Finally BingO!

day/date

Cut Rod (Rewsive)




```
if n==0
    return 0
q = -∞ ⇒ ✓
for i=1→n
    q = max(q, p[i] + CutRod(p, n-i))
return q
```

★

Maybe Not cutting at all is Better forme.

★ When i=n

$p[i] + cutRod(p, 0)$ → returns 0

→ returns value with no cut!

Now let's Introduce Dp:-

(Top down)

★ Make an array rodcut

```
r[0...n]
for i 1→n
    r[i] = -∞
return rodCutMem( )
```

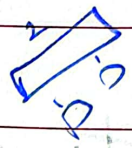
```
rodcutMem ( )
    if r[n] ≥ 0
        return r[n]
    if n==0
        q=0
    else
        q=-∞
```

```
for 1→1→n
    q = max(q, p[i]+...)   n-i
r[n] = q
return q.
```

Finally BingO!

day/date

Bottom Up Approach!



```
r[0...n]
r[0] = 0
for j = 1 → n
    q = -∞
    for i → 1 to j
        q = max(q, p[i] + r[j-i])
    r[j] = q
return r[n]
```

Take the rod to be of size 1 at start then 2, 3 ... n

If j=2 → rod is of 2 size

Then you can either put cut at pos=1 or pos=2 ↳ no cut.

Both have $O(n^2)$ Time Comp.

Extended Version:-

★ Make another array $S[0...n]$

Some code above.

```
if q < p[i] + r[j-i]
    q = p[i] + r[j-i]
    S[j] = i
```

S[j] = i → when size was j → we placed the cut at i!

Two Properties:-

1) Optimal Substructure → If you can find sol of problem by finding optimal sol of subproblem
2) Overlapping Subproblem!

```
While n > 0
    print S[n]
    n = n - S[n]
```

Finally BingO!

day/date

Matrix Chain Multiplication:-

★ We have 4 Matrices $A_1\ A_2\ A_3\ A_4$

5 ways

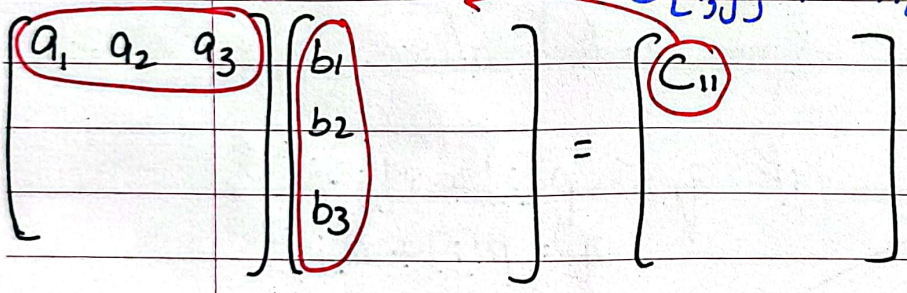
k=1	1) $(A_1(A_2(A_3A_4)))$
k=1	2) $(A_1((A_2A_3)A_4))$
k=2	3) $((A_1A_2)(A_3A_4))$
k=3	4) $((A_1(A_2A_3))A_4)$
k=3	5) $(((A_1A_2)A_3)A_4)$

Multiply Matrices

$A_{pxq}, B_{q,xr}$

→ Ans => C_{pxr}

```
for i → 1 to p
    for j → 1 to r
        for k → 1 to q
            C[i,j] += A[i,k] * B[k,j]
```



★ Q:- How many total Mul did we do?

Ans:- Jitna loops Choday → pxq,xr total Iterations

Brute force checking → Exponential time!

Finally BingO

day/date

First let's create a subproblem:-

* Min case is of 2 $(A_{p \times q}\ B_{q \times r}) \Rightarrow$ Base Case
  Cannot do anyth
* If we have 3 $((A\ B)C)$ OR $(A\ (BC))$
* So how can we write it in terms of optimal problems!

Let $m[i \ldots j]$ be array (dp) → store Best solutions for $A_{i \ldots j}$

Our Problem → $A_{1 \ldots n}$ ⇒ We need to break somewhere in the middle, let's call that k!

So →

$$A_{i \ldots j} = (A_i A_{i+1} \cdots A_k) \cdot (A_{k+1} A_{k+2} \cdots A_j)$$

$$\text{Cost } A_{i \ldots j} = \text{cost}(A_{i \ldots k}) + \text{Cost}(A_{k+1 \ldots j}) + (A_{i \ldots k} * A_{k+1 \ldots}$$

to visualise ⇒ $((AB) * C) \rightarrow A_{1 \ldots 2} * A_{3 \ldots 3}$

$(D * E)$

Now look at m array

$$m[i \ldots j] = m[i \ldots k] + m[k+1 \ldots j] + P_{i-1} P_k P_j$$

* $m[1 \ldots i] = 0$

Visualise

$(A_{pq} (BC)_{qr\ rs})$

$(A_{pq} D_{q,s})$

Finally BingO!

day/date

Recursive Definition:-

$$m[i,j] = \begin{cases} 0 & i == j \\ \min\{m[i,k] + m[k+1,j] + p_{i-1} p_k p_j & i < j \end{cases}$$

$i = 1, 2, \dots n$

$i \le k < j$

all values of k

What are you taking min of?

$$\text{Min} \begin{cases} m[i,0] + m[2+j] + \dots \\ m[i,2] + m[3,j] + \dots \\ \vdots \\ m[i,j-1] + m[j,j] + \dots \end{cases}$$

★ How do we know where the ws are?

Ans:- Use another table $s[i,j] = k$ → $A_{i \dots j}$

Let's run on example first before coding:-

A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

	1	2	3	4	5	6
1	0	15,750	7875			
2	–	0	2,625			
3	–	–	0	750		
4	–	–	–	0	1000	
5	–	–	–	–	0	5000
6	–	–	–	–	–	0

Uses this since smaller

$A_1 (A_2 A_3)$ → 2625 + 5250 = 7875

$A_1 (0)$ → $30 \times 35 \times 5$ → 5250

Finally BingO!

day/date

Code (Bottom Up)

Time $O(n^3)$

Space $O(n^2)$

```
for i -> 1 to n  ---> chains of length = 1
    m[i,i] = 0
for l <- 2 to n  ---> chains length 2,3,4,...n
    for i <- 1 to n-l+1
        j <- i+l-1
        m[i,j] = ∞
        for k <- i to j-1
            q <- m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]
            if q < m[i,j]
                m[i,j] <- q
                s[i,j] <- k
```

return m and s

- Min cost table
- Split table

Print Optimal :-

Using s array for $1 \to n$

$(1, s[1,n])$ → first cut

$(s[1,n], n)$

Recursively

```
Printer(s, i, s[i,j])
Printer(s, s[i,j], j)
```

Finally BingO!

day/date

Recursive Matrix Chain (Bad)

```
if i == j
    return 0
m[i,j] = ∞
for k ← i to j-1
        q ← RMC(p,i,k) + RMC(p,k+1,j) + p_{i-1} p_k p_j
        if q < m[i,j]
            m[i,j] ← q
return m[i,j]
```

Top down (Memoised) :-

```
Initialise m ← ∞
return RMC(p,1,n)

RMC(){

if m[i,j] < ∞
    return ...
if i == j
    return m[i,j] ← 0
else
    for k ← i to j-1
        { Some
```

Finally BingO!

day/date

Lowest/Longest common Subsequence!

★ finding common with Max length.
(Need not consecutive, but in order)!

Naive

$O(n2^m)$ ⇒ for every subseq of X check in Y.

Total → 2^m Subsequences.
→ They each taken n time to check

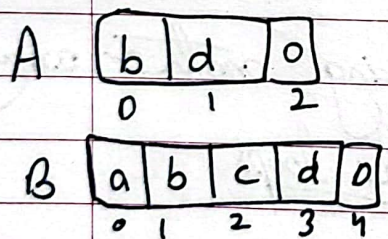
Example:-

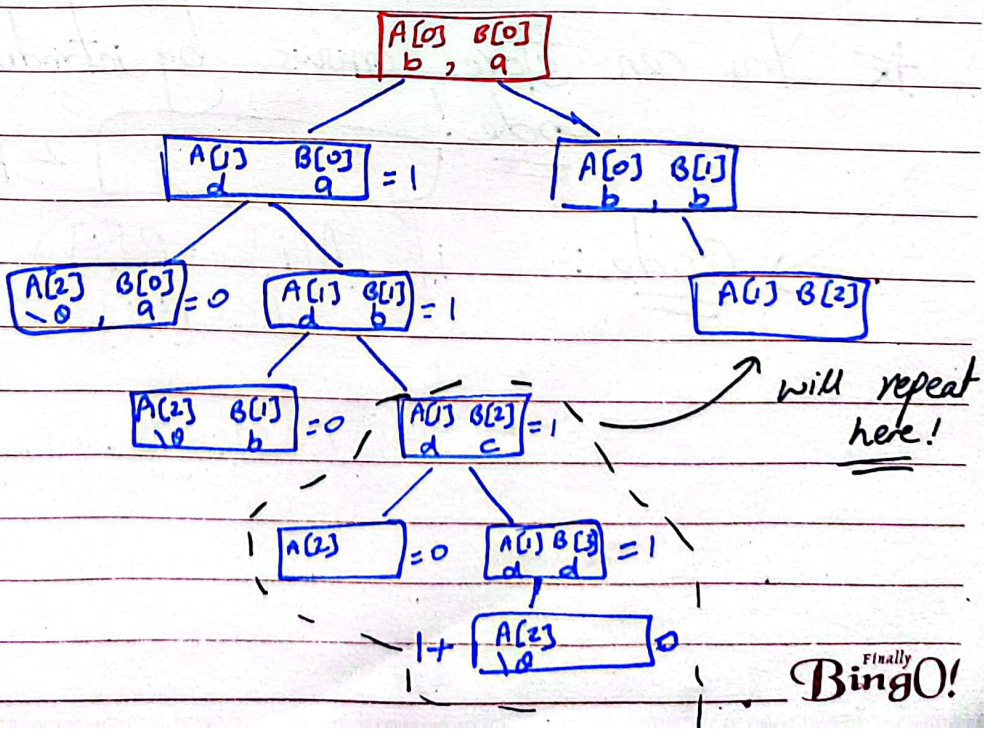
Rewrsive

Top down Approach

Exponential!

```
LCS(i,j)
  if (A[i] == '\0' || B[j] == '\0')
      return 0
  else if (A[i] == B[j])
      return 1 + LCS(i+1, j+1);
  else
      return max(LCS(i+1, j), LCS(i, j+1));
```





Finally BingO!

day/date

		a 0	b 1	c 2	d 3	a 4
b	0	2	2			
d	1	1	1	1	1	
a	2	0	0	0		0

$O(m*n)$ ★

Dynamic Programming :-

	A	b	d		
	B	a	b	c	d
		a	b	ac	d
		0	1	2	3	4
	0	0	0	0	0	0
b	1	0	0	1	1	1
d	2	0	0	1	1	2

Arrows help go back!

Storing Diagonal ~~landing~~ Pos are your answer

b d

bd

Row = 0, col = 0 → Init = 0

★ You can store arrows by introducing another array in code!

2 for loops

Code :-

```
if (A[i] == B[j])
    LCS[i,j] = 1 + LCS[i-1, j-1]
else
    LCS[i,j] = max{LCS(i-1,j), LCS(i,j-1)}
```

Finally BingO!

day/date

To store arrows.

```
if (matching)
        LCS[i,j] = 1 + LCS[i-1, j-1];   ⟶ b[i,j] = '↖'
else if (LCS[i-1, j] ≥ LCS[i, j-1])

        LCS[i,j] = LCS[i-1, j]
        b[i,j] = ↑

[  '←'
```

Printing :-

Print LCS (Recursive)

```
if ('↖')
        PrintLCS (i-1, j-1)
        print x_i
else (↑)
        PLCS (i-1, j)
else (←)
        PLCS (i, j-1)
```

Finally BingO!

day/date

Optimal Binary Search Tree!

BST:-

left hand of node is all smaller.
Right hand all larger.

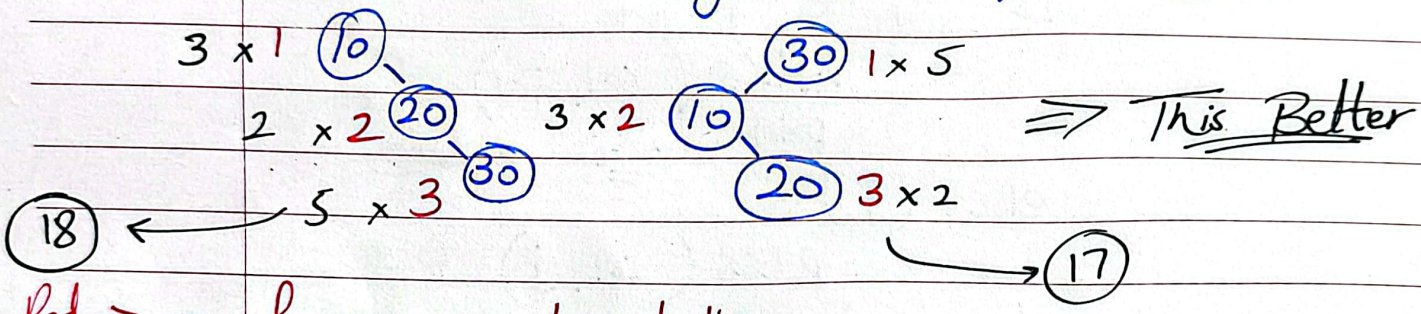
→ Time O(logn) Searching Average

→ Worst case O(n)

Now

OBST:- Special
Take frequency of searching each key to account!

eg:- 10, 20, 30 keys. ⇒ freq are 3, 2, 5



⇒ This Better

Red ⇒ no of comparisons to get there.
black ⇒ freq

★ So optimal sol need not be lowest height!

3 Tables:-

- Probabilities
- Cost
- Root

Finally BingO!

day/date

Total n keys in question in sorted order.

$k_1, k_2 \dots k_n \Rightarrow p_1, p_2 \dots p_n \Rightarrow$ Probability also given

★ $n+1 \Rightarrow$ dummy keys $\Rightarrow$ $d_0, d_1 \dots d_n$ (leaf nodes)

d_0 do all keys less than k_1

d_n all greater than k_n

So:-

$$\sum_{i=1}^{n} p_i + \sum_{i=0}^{n} q_i = 1$$ Sum of all sucess & fails!

Now if just multiply by depth (p and q) we can get cost of searching!

Depth of root is 0 so at root takes (depth +1) comparison.

So Cost is →

$$\sum_{i=0}^{n} p_i \cdot (\text{depth}(k_i)+1) + \sum_{i=0}^{n} q_i \cdot (\text{depth}(d_i)+1)$$

↳ Expand you'll get

$$\Rightarrow 1 + \sum_{i=1}^{n} p_i (\text{depth}(k_i)) + \sum_{i=0}^{n} q_i (\text{depth}(d_i))$$

So the recursive formula for OBST is:-

★ $$\text{optcost}(i,j) = \sum_{k=i}^{j} \text{freq}[k] + \min_{r=i}^{j} [\text{optcost}(i, r-1) + \text{optcost}(r+1, j)]$$

Note to understand this Monstrosity!

Multiplying (depth (k_i)) $p_i \Rightarrow$ adding $p_i + p_i + p_i + \dots$ depth

Finally BingO!

day/date

$$\min\left[\text{optcost}(i,r-1) + \text{optcost}(r+1,j)\right]$$ → Easy to understand this part!

★ We are taking each node as root and calculating cost & returning min amongst all.

$\sum_{k=i}^{j} freq[k]$ → freq(r) + freq(i to r-1) + freq(r+1 to j)

★ freq of all other keys added to account for their increased depth! (one lvl deeper)!

optcost(i, r-1) ⟹ when we make this call. It thinks I have to figure out root. It thinks I am at top! But its one lvl deeper!

So we add its freq once before to accomodate its movement 1 level down

Recursive Code :-

```
optcost (freq[], i, j)
  if (j<i)
      return 0;
  if j == i
      return freq[i]

  fsum = Sum (freq i→j)
  min = ∞
  for (r=i → j)
      cost = optcost(freq, i, r-1) + optcost(freq, r+1, j)
      if(cost < min)
          min = cost;
```

When only Successful hits!

No Unsuccessful!

Bingo!

day/date

Dynamic Programming -:(Solution):-

$i \geq 1$, $i-1 \leq j \leq n$

When $j = i-1$; means (no actual keys) only dummy d_{i-1}

So $e[i, i-1] = q_{i-1}$

So Case 1: $e[i,j] = \min \{ e[i, r-1] + e[r+1, j] \} + W(i,j)$

Sum of all freq from $i \to j$

Case 2:-

$e[i, i-1] = q_{i-1}$

To keep track of structure of OBST → root[i,j]

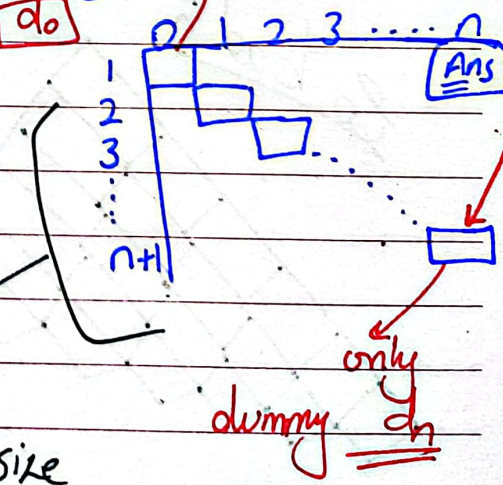
k_r root for $k_i \dots k_j$

Index ranges for $e[1 \dots n+1, 0 \dots n]$

$i \to 1$ to $n+1$ → so that we can store $e[n+1, n]$

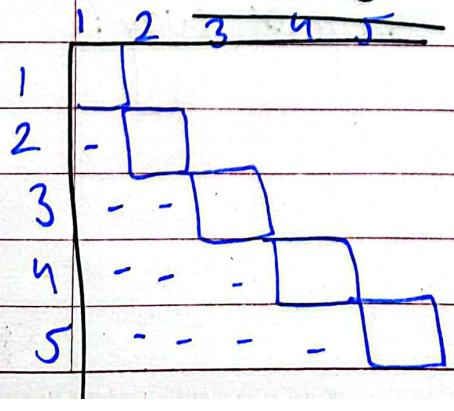
$j \to 0$ to n → so to store $e[1,0]$

final answer $e[1,n]$



Root table:-

$1 \leq i \leq j \leq n$



Finally BingO!

day/date

$$W[i,j] = w[i,j-1] + p_j + q_j$$

Relate

Some

$$W[0,2] = q_0 + p_1 + q_1 + p_2 + q_2$$
$$W[0,3] = q_0 + p_1 + q_1 + p_2 + q_2 + p_3 + q_3$$

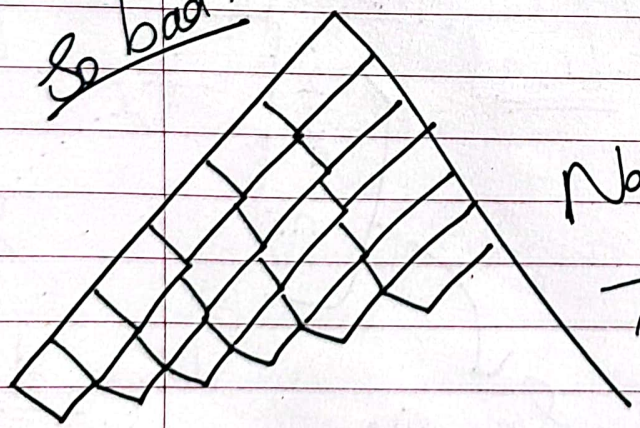
Optimal Algo

```
for i ← 1 to n+1
    e[i,i-1] ← q_{i-1}
    w[i,i-1] ← q_{i-1}

for l ← 1 to n          → Starting from length 1
    for i ← 1 to n-l+1
        j ← i+l-1
        e[i,j] = ∞
        w[i,j] ← w[i,j-1] + p_j + q_j
        for r ← i to j
            t ← e[i,r-1] + e[r+1,j] + w[i,j]
            if t < e[i,j]
                e[i,j] ← t
                root[i,j] ← r
```

So initially we fill
e[1,1]
e[2,2]
⋮

So bad :(



Normally we display like this!

There's another way!

Finally BingO!

day/date

	0	1	2	3	4	5	6
j-i = 0	w_{10}	w_{21}	w_{32}	w_{43}	w_{54}	w_{65} →	
j-i = 1	w_{11}	w_{22}	w_{33}	w_{44}	w_{55}		
j-i = 2	w_{12}	w_{23}	w_{34}	w_{45}			
j-i = 3	w_{13}	w_{24}	w_{35}				
j-i = 4	w_{14}	w_{25}					
j-i = 5	w_{15}						

first diagonal
2nd
3rd
⋮

k is r

$$c[i,j] = \min_{i \le k \le j} \{ e[i,k-1] + e[k+1,j] \} + w(i,j)$$

Example Practice

keys = {10, 20, 30, 40}
p_i = {3, 3, 1, 1}
q_i = {2, 3, 1, 1, 1}

whole nums to simplify working!

	1	2	3	4	5
0	$w_{10} \to 2$ $c_{10} \to 0$ $r_{10} \to 0$	$w_{21} \to 3$ $c_{21} \to 0$ $r_{21} \to 0$	$w_{32} \to 1$ $c_{32} \to 0$ $r_{32} \to 0$	$w_{43} \to 1$ $c_{43} \to 0$ $r_{43} \to 0$	$w_{54} \to 1$ $c_{54} \to 0$ $r_{54} \to 0$
1	$w_{11} \to 8$ $c_{11} \to 8$ $r_{11} \to 1$	$w_{22} \to 7$ $c_{22} \to 7$ $r_{22} \to 2$	$w_{33} \to 3$ $c_{33} \to 3$ $r_{33} \to 3$	$w_{44} \to 3$ $c_{44} \to 3$ $r_{44} \to 4$	
2	$w_{12} \to 12$ $c_{12} \to 19$ $r_{12} \to 1$	$w_{23} \to 9$ $c_{23} \to 12$ $r_{23} \to 2$	$w_{34} \to 5$ $c_{34} \to 8$ $r_{34} \to 3$		
3	$w_{13} \to 14$ $c_{13} \to 25$ $r_{13} \to 2$	$w_{24} \to 11$ $c_{24} \to 19$ $r_{24} \to 2$			
4	$w_{14} \to 16$ $c_{14} \to 32$ $r_{14} \to 2$				
5					

	0	1	2	3	4
1	0				
2	-	0			
3	-	-	0		
4	-	-	-	0	
5	-	-	-	-	0

$w[i,0] = q_0$
$w[2,1] = q_1$

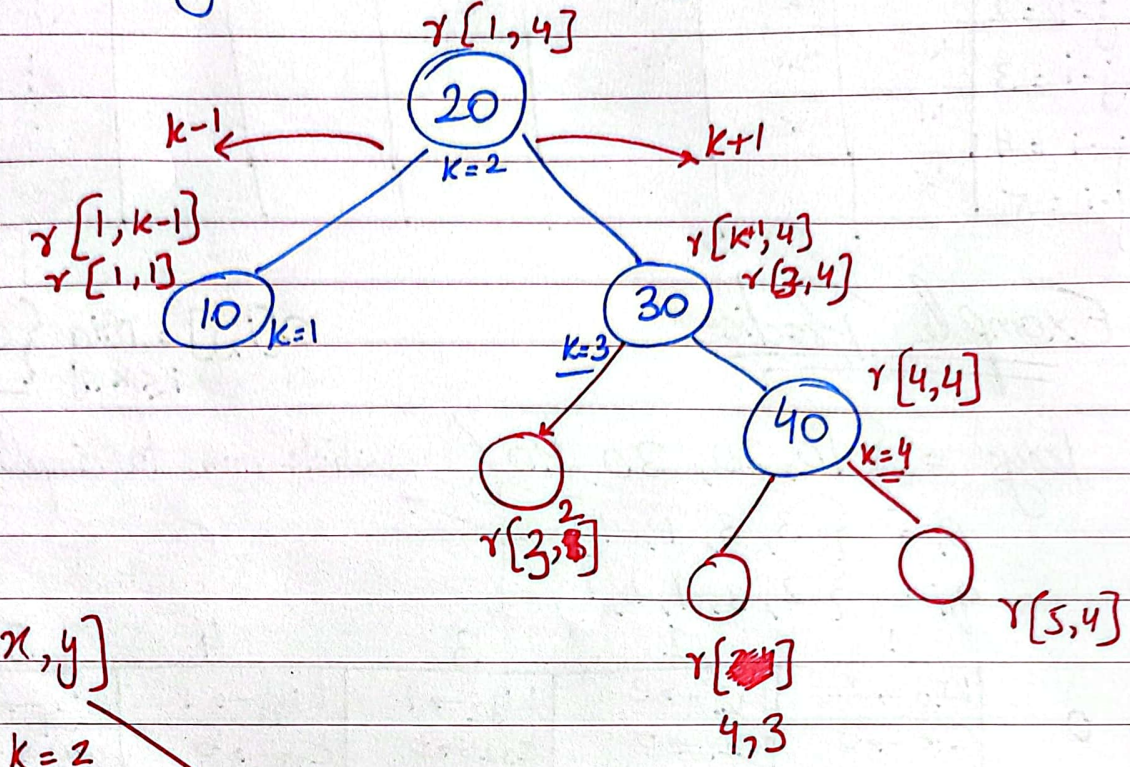
$w[i,j] = w[i,j-1] + p_j + q_j$

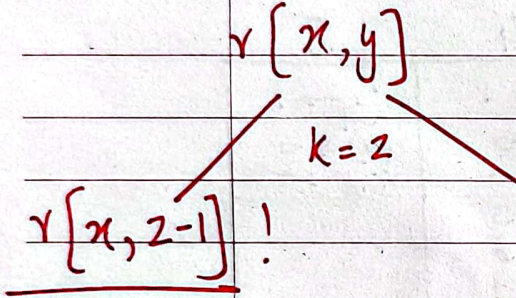
Finally BingO!

day/date

So how to construct the tree :-

Root finally we see root is **2**





Finally BingO!

day/date

0/1 Knapsack Problem:-

★ Already done fractional using greedy method.

Let's solve:-

Brute force! → all subsets

If n items, 2^n ways $O(2^n)$

Find Combination with weight $< W$ & Max benefit

1 approach:- $S_k = \{$ items 1, 2 ... k $\}$

→ may or may not have all items from 1→k

→ Is this a Proper Subproblem?

Ans:- no. S_5 should have S_4 in its part of solution not the case!

	W	B
1	2	3
2	3	4
3	4	5
4	5	8
5	9	10

for S_4 → [1,2,3,4]

for S_5 → [1,3,4,5]

W=20

Need to redefine Subproblem:-

Let's add w parameter → Total weight of subset of items

So

$$B[k,w] = \begin{cases} ① \; B[k-1,w] & \text{if } w_k > w \\ ② \; \max\{B[k-1,w], B[k-1, w-w_k] + b_k\} & \text{else} \end{cases}$$

Finally BingO!

day/date

Explanation:-

① Since adding this current item exceeds weight, use the previous solution without this item.

② Either choose without this item or with this item whichever is Max]

* but if you choose this item. You need to add $B[k-1, w-w_k]$ Best sol without this item and when we have max $w-w_k$ weight limit

Code:

```
for w = 0 to W
    B[0,w] = 0.
for i = 1 to n
    B[i,0] = 0

for i = 1 to n
    for w = 0 to W
        if wi <= w
            if bi + B[i-1, w-wi] > B[i-1, w]
                B[i,w] = bi + B[i-1, w-wi]
            else
                B[i,w] = B[i-1, w]
        else
            B[i,w] = B[i-1, w]   // wi > w *
```

O(nw)

Finally BingO!

day/date

$n = 4$ Elements (w, B)

$W = 5$ $(2,3), (3,4), (4,5), (5,6)$

i \ W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7
✗						

Don't make item 3 a Part
Better solution without it

How to know which items are included in knapsack?

Ans:-

```
i=n , j=m(W)
while (i>0 && j>0){
    if (B[i][j] == B[i-1][j])
        i is not included; i--;
    else
        i included; i--; j=j-wt[i];
}
```

Finally BingO!

day/date

(Hashing) <Quick Revision>

Worst case O(n)
Best O(1)

So :- A Table of size m → (0...m-1)

T[i] = x

key[x] = i → Hash ftn ★

Direct Address Table

2 Stored at index 2 -: Takes 1 step :-
10 at index 10 → Bad

Problems:-

1) Waste space if U large but actually we are storing less keys!

Hash/Mapping ftn:- → Collision Problem

Sol

1) Chaining ⟹ Best/Avg O(1) ⇒ Worst O(n)
2) Open addressing

Chaining → If equally likely each key to any slot!

n → keys m → slots. load = n/m keys/slot → α

O(1+α) → Cost of search

Finally BingO

day/date

Nature of Keys:-

If characters then pa, ap same for sum ASCII

Sol

Use → 128-radix notation → pf ⇒ (112·128) + 116

128^2 128^1 128^0

Rules:-

1) fully determined by data
2) Uses all Input data
3) Uniformly distributes n → m
4) D/f hashes for similar strings!

Analyse Sum ASCII:-

1) Satisfied → fully Yes by all data input.
2) Satisfied ⇒ all are used
3) X No Uniform
4) X CAT, ACT

Division Method:-

n → keys
m → slots

1) key must integer
   ↳ (0 ... m-1)

$k \bmod m$

m not power of 2

If $m = 8$, $2^3 \Rightarrow 2^p$

then only last p bits determine data.

Should depend on all bits / key

Avoid power of 10

* Primes ✓
  ↳ Away from 2^p and 10^p

Finally BingO!

day/date

Multiplication (Method)

$h(k) = \lfloor m(kA \bmod 1) \rfloor$ floor

$kA \bmod 1 \Rightarrow kA - \lfloor kA \rfloor$

↳ Extracting fractional Part!

$A \rightarrow [0, 1)$ range

↳ Good = 0.618

Open Addressing :-

All keys in table

Probing (keep going till empty)!

To search follow same method

★ Deletion issue ⇒ Can't normally delete

need to mark as removed otherwise during searching won't get correct result.

Pre emptive termination of search!

First map key → $i_0 = h_1(k)$ → Index where we place

If collision →

→ Index where we had collision

$i_{j+1} = (i_j + c) \bmod m$

→ Should be prime too.

Finally BingO!

day/date

Insertion :-

```
i ← 0
repeat j ← h(k,i)
    if T[j] = NIL
        T[j] = k
        return j
    else
        i ← i+1
until i = m
```

Search :-

```
i ← 0
repeat j ← h(k,i)
    if T[j] = k
        return j
    else
        i ← i+1
until T[j] = NIL or i = m
```

Without deletion

Linear Prob :-

i_0 → first → home position.

Several insert same ↗ cluster

↓

If grows → Prob of key mapping to ↑

Sol :-

Quadratic Probing or Double hashing

→ Secondary Clustering (Milder)

$$h(k,i) = (h(k,0) + c_1 i + c_2 i^2) \bmod m$$

Initial hash

i means how many times have you tried again? ✓

Using another hash fn stead of c

$$i_{j+1} = (i_j + h_2(k)) \bmod m$$

Finally BingO!

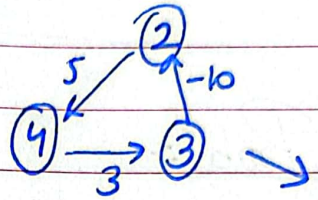
day/date

Bellman Ford Algo :-

Single source Shortest path with negative weights!!

* It fails when there is a cycle with total weight negative

eg



5-10+3 = -2

but if this sum is + no problem!

Code:-

Algorithm:-

source

```
d[s] = 0
for each v ∈ V - {s}
    d[v] = ∞

for i ← 1 to |V| - 1
    for each edge (u,v) ∈ E
        if d[v] > d[u] + w(u,v)
            d[v] = d[u] + w(u,v)
```

relaxation step

To check if bellman failed due to neg weight cycle

```
for each edge (u,v) ∈ E
    if
        True (exists cycle)
```

relax one more!

Time (V-1)E

$O(VE)$

if $V \to n$, $E \to n$

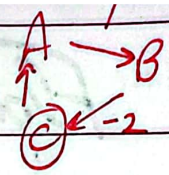
$O(n^2)$

b/w every pair vertices

if complete graph ⇒ edge $\left[\frac{n(n-1)}{2}\right](n-1) \Rightarrow n^3$

Finally BingO!

day/date



Why are we repeating |V|-1 times?

Ans:- After first iteration, shortest path reachable in 1 step calculated

→ 2nd, in 2 steps.

→ So go till |V|-1 since that's the longest possible path.

Note

You can take any sequence of edges to relax Answer some

☆ Dijkstra fails because of its greedy nature

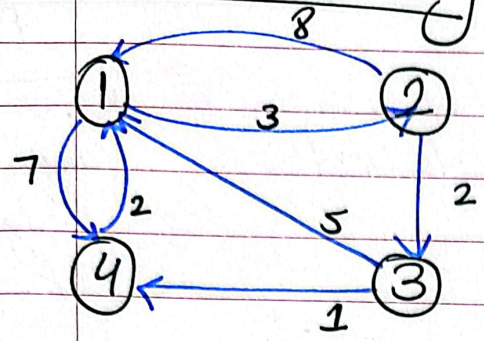
→ It does not process those vertices that have already been processed before! So it fails!

Finally BingO!

day/date

Floyd Warshall (All pair Shortest path)

Let's understand using an example :-



So need to find all

$1 \leq \begin{matrix}2\\3\\4\end{matrix}$ $3 \leq \begin{matrix}1\\2\\4\end{matrix}$

$2 \leq \begin{matrix}1\\3\\4\end{matrix}$ $4 \leq \begin{matrix}1\\2\\3\end{matrix}$

If we use dijkstra for all vertices 1 by 1 ?

Ans :- $(n^2) \times n \Rightarrow n^3$ Yes Possible

Basic

no intermediaries only Direct

$A =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

If there is no direct ∞

1 → 1 ⇒ 0

same

Shorter paths going via vertex 1.

Vertex 1 as intermediate

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	8	∞	0

1st row & col comes as it is!

↳ 1 is intermediate not start or end

For each calculation do this

$A^0[2,3]$ or $A^0[2,1] + A^0[1,3]$

(2) (8 + ∞)

Min

Finally BingO!

day/date

Same remains

A^2 =

	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	8	7	0

$A^1[1,3]$ or $A^1[1,2] + A^1[2,3]$

∞ $3 + 2$

A^3 =

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

$A^2[1,2]$ or $A^2[1,3] + A^2[3,2]$

3 $5 + 8$

A^4 =

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Formula:-

$$A^k[i,j] = \min\left\{A^{k-1}[i,j],\ A^{k-1}[i,k] + A^{k-1}[k,j]\right\}$$

$O(V^3)$

Code:-

```
for k=1 to n
    for u in V
        for v in V
            if Cuv > Cuk + Ckv
                Cuv = Cuk + Ckv
```

Finally BingO!

day/date

```
for k 1→n
    for i 1→n
        for j 1→n
            A[i,j] = min{ A^(k-1)[i,j], A^(k-1)[i,k] + A^(k-1)[k,j] }
```

$d_{ij}^{(k)}$ ⇒ Shortest distance b/w i and j using $\{1, 2, \ldots k\}$ as intermediate vertices.

Dynamic tables:-

Make table small but large enough so no overflow!

Initially Size 1 → 2 → 4 → 8 → 16 → 32.

What is the worst Case (Insertion?)

$$Cost = \begin{cases} i & \text{if } i-1 \text{ is power of 2} \\ 1 & \text{else} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
size	1	2	4	4	8	8	8	8	16
c_i	1	2	3	1	5	1	1	1	9
	1	1	1	1	1	1	1	1	1
	D1	2	0	4	0	0	0	8	

So cost is for n insertions $\sum_{i=1}^{n} c_i \leq n + \sum_{j=0}^{(\log n - 1)} 2^j \leq 3n$

$O(n)$

→ Avg cost for 1 Insertion = $O(1)$

Finally BingO!

day/date

Amortised Analysis :-

Strategy = Analyse seq of ops to show Avg cost per op is small!

Avg performance of each op in Worst case!

3 Methods
i) Aggregate
ii) Accounting
iii) Potential

Accounting Method! :-

charge i^{th} op fictitious amortized cost. → $\hat{c}_i$

fee consumed → remaing bank!

★ Imp
Bank never negative!

i	1	2	3	4	5	6	7	8	9	10
Size	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2*	3	3	3	3	3	3	3	3	3
bank	1	2	2	4	2	4	6	8	2	4

first op 2

★ Since sequence of ops and we know the some ops are cheaper other expensive. So at cheap operation we save power & utilise that at expensive op!

Finally BingO!

day/date

Potential Method!

$D_0 \Rightarrow$ Initial data structure

op i $\rightarrow$ changes $D_{i-1} \rightarrow D_i$

cost $i \rightarrow c_i$

Potential Fn - denoted by Φ

$\Phi(D_0) = 0 \rightarrow$ Initially 0 potential

$\Phi(D_i) \geq 0$ for all i

So finally $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$

$\hat{c}_i$ — amortized cost; c_i — normal; $\Delta\Phi_i \rightarrow$ potential diff

$\Delta\Phi_i > 0 \Rightarrow \hat{c}_i > c_i$ (We store energy)

$\Delta\Phi_i < 0 \Rightarrow \hat{c}_i < c_i$ (release energy)

Total Amortized cost $\rightarrow$ $\sum_{i=1}^{n} \hat{c}_i \Rightarrow \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1}))$

$\sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$

$\Phi(D_0) = 0$

Finally BingO!

day/date

Let's define a formula for

$$\phi(D_i) = 2i - 2^{\lceil \lg i \rceil}$$

$$2^{\lceil \lg 0 \rceil} = 0$$

$$\Delta\phi_i \Rightarrow 2 - 2^{\lceil \lg i \rceil} + 2^{\lceil \lg (i-1) \rceil} \leftarrow \phi(D_i) - \phi(D_{i-1})$$

Expand

Case 1: $i-1$ exact power of 2

$$\hat{c}_i = i + 2 - 2^{\lceil \lg i \rceil} + 2^{\lceil \lg (i-1) \rceil}$$

$$= i + 2 - 2(i-1) + (i-1)$$

$$= 3$$

can be shown

let $i-1 = 2^3$

$2^{\lceil \log_2 (2^3) \rceil}$

$2^3 \Rightarrow (i-1)$

$i = 2^3 + 1$

$2^{\lceil \log_2 (2^3+1) \rceil}$

$2^4 \rightarrow 2^3 \cdot 2$

$(i-1)2$

Case 2 → also 3

So $\hat{c}_i = 3$

Finally BingO!